# Unit 2
# Data Representation & Compression

*100-Question Priority-Starred Master Set*

- Questions 1–90: Multiple Choice (Full Priority Ratings & Distractor Breakdowns)
- Questions 91–100: Create Performance Task Style Written Responses (9 Points Each)
- Complete Answer Key and Detailed Worked Solutions at the End

# Examination Structure

This paper contains 100 continuously numbered questions designed for complete mastery of AP Computer Science Principles Unit 2: Data Representation & Compression (Big Idea 2: DAT, weighted 17%–22% of the AP Exam). All answers, step-by-step worked solutions, and 9-point scoring rubrics appear after Question 100.

### ★ Priority Rating System

| | |
|---|---|
| ★★★★★ **Very Important** | High-frequency exam concepts: Binary-to-decimal & decimal-to-binary conversions, Bit doubling rule ($2^n$), Integer overflow mechanics (rollover & wrap-around), Floating-point round-off errors (repeating 0.1 in binary), Analog continuous vs. digital discrete data, Audio sampling rate & bit depth formulas, Lossless vs. Lossy compression trade-offs, Metadata definition & privacy risks, Correlation vs. Causation, and Sampling bias in datasets. |
| ★★★★ **High Priority** | Core computational standards: Hexadecimal base-16 conversions (4 bits per nibble), RGB additive color mixing (#RRGGBB), Run-Length Encoding (RLE) efficiency, Dictionary (LZW) compression mechanics, Perceptual psychovisuals & psychoacoustics, Compression ratio calculations, EXIF photo headers, Data cleaning (deduplication & outlier detection), and Parallel cluster processing. |
| ★★★ **Mandatory** | Essential computing mechanics: Binary addition column carries, Odd/even parity (least significant bit), ASCII vs. Unicode (UTF-8), String characters vs. numeric integers in RAM, Truncation vs. rounding, Digital generation loss immunity, and Handling missing data. |

### Topic Coverage (Big Idea 2: DAT-1 & DAT-2)

**2.1** Binary Numbers (Place Values, Conversions, Range, Powers of 2)

**2.2** Hexadecimal & Text Encoding (Hex-to-Binary, RGB Colors, ASCII, Unicode)

**2.3** Integer Overflow & Bit Limits (Fixed-Width Registers, Two's Complement)

**2.4** Floating-Point & Round-Off (Repeating Fractions, Equality Bugs, Epsilon)

**2.5** Analog vs. Digital & Sampling (Sampling Rate, Bit Depth, Quantization)

**2.6** Lossless Compression (Run-Length Encoding, Dictionary/LZW, Ratios)

**2.7** Lossy Compression (Perceptual Discarding, Artifacts, Irreversibility)

**2.8** Metadata & Privacy (Data About Data, EXIF, Sanitization, Tracking)

**2.9** Data Cleaning & Big Data (Outliers, Deduplication, Correlation vs. Causation)

> **AP Exam Pro Tip:** Remember: *Integer Overflow* occurs with whole numbers when an arithmetic sum exceeds the maximum value representable by fixed register bits (e.g. 255 rolls to 0). In contrast, *Round-Off Error* occurs with real fractional numbers because fractions like 0.1 cannot be represented with finite binary digits! In compression, remember that **lossless** is 100% reversible, while **lossy** permanently discards data!

- **Questions 1–90:** Multiple choice, four options (A–D), exactly one correct answer.
- **Questions 91–100:** Create Performance Task Written Response scenarios, 9 points each, with full rubrics.
- **Difficulty Mix:** Authentic Medium AP Exam Level ❖ Rigorously 3-Pass Verified.
- **Answer Section:** Begins after Question 100 — master answer key, detailed MCQ solutions with distractor failure analysis, and model FRQ solutions with official point allocations.

ITEM 01

TOPIC: BINARY TO DECIMAL CONVERSION ★★★★★ VERY IMPORTANT

A network router displays a device's local subnet identifier as the 8-bit binary number $10100110_2$. What is the decimal (base-10) equivalent of this binary value?

**A** 166
**B** 162
**C** 150
**D** 174

ITEM 02

TOPIC: DECIMAL TO BINARY CONVERSION ★★★★★ VERY IMPORTANT

A microcomputer system records a sensor reading of 89 in base-10. What is the 8-bit binary representation of 89?

**A** 01001101
**B** 01101001
**C** 01011001
**D** 01011011

ITEM 03

TOPIC: NUMBER OF VALUES REPRESENTED BY N BITS ★★★★★ VERY IMPORTANT

A software developer increases the user ID field in a database from 6 bits to 9 bits. How many more distinct user IDs can be uniquely represented after this change?

**A** 512 more user IDs
**B** 3 more user IDs
**C** 64 more user IDs
**D** 448 more user IDs

ITEM 04

TOPIC: MINIMUM BITS REQUIRED FOR TARGET RANGE

★★★★★ VERY IMPORTANT

A school attendance system needs to assign a unique binary identification number to every student currently enrolled. If the school has 1,250 students, what is the minimum number of bits required to ensure every student receives a unique binary code?

**A** 8 bits

**B** 10 bits

**C** 12 bits

**D** 11 bits

ITEM 05

TOPIC: BIT PLACE VALUES AND DOUBLING PRINCIPLE

★★★★ HIGH PRIORITY

Which of the following statements correctly describes what happens to the total number of distinct numerical values that can be represented whenever one additional bit is added to a binary sequence?

**A** The total number of representable values doubles.

**B** The total number of representable values squares.

**C** The total number of representable values increases by 2.

**D** The maximum representable value increases by 1.

ITEM 06

TOPIC: BINARY NUMBER COMPARISON

★★★ MANDATORY

Consider the following three binary numbers:

- $P = 01101100_2$
- $Q = 01110001_2$
- $R = 01101111_2$

Which of the following correctly orders these values from least to greatest?

**A** P < Q < R

**B** Q < P < R

**C** P < R < Q

**D** R < P < Q

ITEM 07

TOPIC: ODD AND EVEN BINARY NUMBERS

★★★★ HIGH PRIORITY

**A student inspects an arbitrary positive binary integer stored in memory. Without converting the entire binary sequence to decimal, which single test reliably determines whether the number is odd or even?**

**A** Examine the leftmost bit (most significant bit): if it is 1, the number is odd; if it is 0, the number is even.

**B** Check whether the number of bits in the sequence is an odd or even integer.

**C** Examine the rightmost bit (least significant bit): if it is 1, the number is odd; if it is 0, the number is even.

**D** Count the total number of 1s in the sequence: an odd count indicates an odd number.

ITEM 08

TOPIC: MAXIMUM UNSIGNED VALUE WITH N BITS

★★★★★ VERY IMPORTANT

**What is the maximum decimal value that can be represented using an unsigned 8-bit binary number?**

**A** 128

**B** 255

**C** 127

**D** 256

ITEM 09

TOPIC: BINARY ADDITION MECHANICS

★★★ MANDATORY

**Consider the binary addition of two 4-bit numbers: $0110_2 + 0101_2$. What is the resulting binary sum?**

**A** $1111_2$

**B** $1100_2$

**C** $1001_2$

**D** $1011_2$

ITEM 10

TOPIC: BIT REPRESENTATION OF DISCRETE STATES

★★★★ HIGH PRIORITY

A robotic warehouse delivery vehicle has 5 distinct operating states: Charging, Idle, Picking, Navigating, and Delivering. What is the minimum number of bits needed to store the vehicle's current state in memory?

**A** 4 bits

**B** 5 bits

**C** 2 bits

**D** 3 bits

ITEM 11

TOPIC: HEXADECIMAL TO BINARY CONVERSION

★★★★★ VERY IMPORTANT

An engineer inspects an error code displayed by an operating system as hexadecimal $4D_{16}$. What is the 8-bit binary equivalent of $4D_{16}$?

**A** 01001110

**B** 00101101

**C** 01001101

**D** 01001011

ITEM 12

TOPIC: BINARY TO HEXADECIMAL CONVERSION

★★★★★ VERY IMPORTANT

A graphics rendering engine stores a pixel transparency channel as the binary value $10111110_2$. What is the hexadecimal equivalent of this binary byte?

**A** $BD_{16}$

**B** $BF_{16}$

**C** $BE_{16}$

**D** $AE_{16}$

ITEM 13

TOPIC: PURPOSE OF HEXADECIMAL IN COMPUTING

★★★★ HIGH PRIORITY

**Which of the following best explains why software developers and network engineers frequently represent binary memory addresses and color codes in hexadecimal rather than binary?**

**A** Hexadecimal is much more concise and human-readable, reducing long sequences of bits while maintaining an exact, direct 1-to-4 mapping with binary.

**B** Modern computer processors execute instructions natively in hexadecimal hardware circuits rather than binary electronic switches.

**C** Hexadecimal format encrypts the underlying data so that unauthorized users cannot inspect memory values.

**D** Hexadecimal numbers can store more total data within the same physical memory silicon than binary numbers.

ITEM 14

TOPIC: RGB HEXADECIMAL COLOR DECODING

★★★★★ VERY IMPORTANT

**A web stylesheet specifies a background color using the 24-bit hexadecimal color code `#00FF00`. Which of the following visual colors does this code display on a screen?**

**A** Pure bright green

**B** Dark gray

**C** Pure bright blue

**D** Pure bright red

ITEM 15

TOPIC: RGB ADDITIVE COLOR MIXING IN HEX

★★★★ HIGH PRIORITY

**In digital screens, colors are generated using additive color mixing. If a UI designer combines full-intensity red and full-intensity green while turning off blue, what hexadecimal color code is produced, and what visual color results?**

**A** `#FFFFFF`, producing white

**B** `#FF00FF`, producing magenta

**C** `#00FFFF`, producing cyan

**D** `#FFFF00`, producing bright yellow

ITEM 16

TOPIC: ASCII CHARACTER ENCODING STANDARDS

★★★★ HIGH PRIORITY

**In the standard 7-bit ASCII encoding table, the uppercase letter 'A' is represented by decimal 65 (binary $1000001_2$). What binary sequence represents the uppercase letter 'C' in standard 7-bit ASCII?**

**A** $1000010_2$

**B** $1000100_2$

**C** $0100011_2$

**D** $1000011_2$

ITEM 17

TOPIC: ASCII VS. UNICODE COMPARISON

★★★★★ VERY IMPORTANT

**Early computing systems relied heavily on ASCII encoding, whereas modern operating systems and web browsers use Unicode (such as UTF-8). Which of the following best describes the primary advantage of Unicode over traditional ASCII?**

**A** Unicode compresses text files so that English characters take 50% fewer bits than standard ASCII.

**B** Unicode completely replaces binary bits with analog signals to render smoother font curves on high-resolution displays.

**C** Unicode uses a variable/larger bit-width capable of representing over a million unique characters, enabling support for all global languages, mathematical symbols, and emojis.

**D** Unicode restricts all international characters to a fixed 7-bit length to maintain strict backward compatibility.

ITEM 18

TOPIC: STRING CHARACTER VS. NUMERIC VALUE REPRESENTATION

★★★ MANDATORY

**In a programming language, the number 5 can be stored as an integer variable `num = 5` or as a text character `char = '5'`. Which of the following statements correctly describes how these two items are stored in computer memory?**

**A** The integer 5 is stored as its binary numeric equivalent ($00000101_2$), while the character '5' is stored as its ASCII/Unicode character code ($00110101_2$).

**B** Both the integer 5 and the character '5' are stored using the identical binary pattern $00000101_2$, with the IDE differentiating them through font formatting.

**C** The character '5' takes 5 bits of memory, while the integer 5 takes 1 bit of memory.

**D** The integer 5 is stored in binary, but text characters are stored directly as vector font graphics in RAM.

ITEM 19

TOPIC: HEXADECIMAL COUNTING SEQUENCE ★★★ MANDATORY

A memory pointer starts at hexadecimal address $2E_{16}$. If the program increments the pointer by 1 twice (+2 in decimal), what is the new hexadecimal address?

**A** $30_{16}$

**B** $2G_{16}$

**C** $2F_{16}$

**D** $31_{16}$

ITEM 20

TOPIC: HEXADECIMAL STRING LENGTH AND BYTE STORAGE ★★★★ HIGH PRIORITY

A secure cryptographic hash function outputs the digest `A3F09C8B`. How many bytes of data does this hexadecimal string represent?

**A** 2 bytes

**B** 4 bytes

**C** 16 bytes

**D** 8 bytes

ITEM 21

TOPIC: DEFINITION AND ROOT CAUSE OF INTEGER OVERFLOW ★★★★★ VERY IMPORTANT

Which of the following best explains why an integer overflow error occurs in computer systems?

**A** The CPU's physical clock frequency fluctuates, causing bits to switch states randomly.

**B** The programmer uses a variable name that has not been initialized in memory, causing a syntax rejection.

**C** A calculation produces a numerical value that exceeds the maximum value that can be represented within the allocated fixed number of bits.

**D** The program attempts to divide a positive integer by zero, causing an arithmetic exception in the ALU.

ITEM 22

TOPIC: 4-BIT REGISTER OVERFLOW MECHANICS

★★★★★ VERY IMPORTANT

**A micro-controller uses an unsigned 4-bit register to track the number of items on an assembly line. The register currently holds the value $1111_2$ (decimal 15). What happens if one more item is detected and the system attempts to add 1 to this register?**

**A** The register automatically expands its physical hardware to 5 bits and safely stores $10000_2$ (decimal 16).

**B** The register holds the value $1111_2$ permanently and ignores all subsequent sensor pulses.

**C** The register crashes the physical microprocessor and requires a complete factory hard-reset.

**D** An overflow occurs, and the register resets to $0000_2$ because the fifth bit cannot be stored in the 4-bit register.

ITEM 23

TOPIC: 8-BIT UNSIGNED INTEGER OVERFLOW

★★★★★ VERY IMPORTANT

**A high-score counter in a vintage 8-bit arcade game currently displays the value 255. When the player earns 1 additional point, the counter rolls over and displays 0 instead of 256. Which of the following explains this behavior?**

**A** The game stores the score in an unsigned 8-bit integer, where 255 ($11111111_2$) is the maximum representable value; adding 1 causes the 9th bit to be discarded, leaving $00000000_2$.

**B** The game engine encountered a floating-point round-off error that rounded 256 down to 0.

**C** The monitor refresh rate could not draw three digits simultaneously on the CRT screen.

**D** The player's input controller sent an illegal ASCII character that wiped the game's high-score memory.

ITEM 24

TOPIC: SIGNED TWO'S COMPLEMENT RANGE AND OVERFLOW

★★★★ HIGH PRIORITY

**A software program allocates an 8-bit signed integer using two's complement notation to store temperature in degrees Celsius. The range of values this variable can store is -128 to +127. If the variable currently holds +127 ($01111111_2$) and the program adds 1, what value does the variable represent?**

**A** -128

**B** 0

**C** +128

**D** -1

ITEM 25

TOPIC: HISTORICAL VIDEO GAME GLITCH AND 8-BIT OVERFLOW

★★★★ HIGH PRIORITY

**In the original 1980 arcade version of Pac-Man, players who successfully completed level 255 encountered a famous 'kill screen' on level 256 where the right side of the fruit board was replaced with corrupted garbled tiles. What programming limitation caused this flaw?**

**A** The game developer deliberately inserted a secret cryptographic key that terminated the game.

**B** An 8-bit level counter variable overflowed when incrementing past 255, causing the subroutine drawing bonus fruit to enter an infinite loop drawing 256 fruit items instead of 7.

**C** The arcade CRT monitor physically burned out after displaying more than 255 consecutive maze screens.

**D** The arcade machine lost internet connection to the global leaderboard database after level 255.

ITEM 26

TOPIC: ARIANE 5 ROCKET FAILURE & TYPE CONVERSION OVERFLOW

★★★★★ VERY IMPORTANT

**In 1996, the European Space Agency's Ariane 5 rocket exploded 37 seconds after liftoff due to a software defect. The guidance computer attempted to convert a 64-bit floating-point measurement of horizontal velocity into a 16-bit signed integer. The value exceeded 32,767, causing an unhandled hardware exception. What computing phenomenon was responsible for this catastrophic failure?**

**A** A syntax error that was ignored by the programming language compiler.

**B** A logic error where the rocket's altimeter was calibrated in miles instead of kilometers.

**C** An integer overflow error during numerical type conversion from a wide data type to a narrow data type.

**D** A lossy compression artifact that permanently removed the rocket's telemetry trajectory data.

ITEM 27

TOPIC: THE YEAR 2038 PROBLEM (UNIX EPOCH OVERFLOW)

★★★★★ VERY IMPORTANT

**Many legacy computer systems store dates and times as the number of elapsed seconds since January 1, 1970 (the Unix Epoch) using a 32-bit signed integer. On January 19, 2038, this counter will reach 2,147,483,647 seconds. What will occur immediately after this point if the system is not upgraded?**

**A** The system will switch to 12-hour AM/PM time and delete all calendar records.

**B** The computer's motherboard clock will run twice as fast to catch up with real time.

**C** The signed 32-bit counter will overflow into negative numbers (-2,147,483,648), causing systems to interpret the date as December 13, 1901.

**D** The system will automatically advance to January 1, 3000 without any interruption.

ITEM 28

TOPIC: PREVENTING INTEGER OVERFLOW DEFECTS ★★★★ HIGH PRIORITY

**A software development team is building a global payment settlement engine where daily transaction totals regularly exceed several billion dollars. Which of the following technical strategies most effectively prevents integer overflow in this financial application?**

**A** Convert all numbers into 7-bit ASCII text strings and perform arithmetic in hexadecimal.

**B** Compress all financial variables using lossy compression before saving them to disk.

**C** Round all transaction amounts to the nearest hundred dollars to keep numbers smaller.

**D** Use 64-bit integers or arbitrary-precision integer libraries ('BigInt') that dynamically allocate memory as numbers grow.

ITEM 29

TOPIC: MECHANICAL ODOMETER ANALOGY FOR OVERFLOW ★★★ MANDATORY

**An automobile odometer has 5 mechanical dials, each showing the digits 0 through 9. The maximum distance it can display is 99,999 miles. When the car travels 1 more mile, the display reads 00,000 miles. How does this mechanical behavior analogize binary computer systems?**

**A** It represents an overflow error in a fixed-width base-10 system, perfectly illustrating how fixed-width registers wrap around when their physical capacity is exceeded.

**B** It demonstrates a round-off error where continuous analog wheel rotation is rounded to zero.

**C** It illustrates lossy compression by discarding the highest 9 to save dashboard space.

**D** It represents a syntax error where the mechanical gears were assembled in the wrong order.

ITEM 30

TOPIC: OVERFLOW VS. ROUND-OFF VS. LOGIC ERRORS ★★★★ HIGH PRIORITY

**Which of the following scenarios describes an **overflow error**, as distinguished from a round-off error or a logic error?**

**A** A retail checkout program calculates sales tax by adding 0.05 instead of multiplying by 0.05.

**B** An uncompressed video file counter adds 1 to an integer variable currently storing $2^{32} - 1$, resulting in an unexpected value of 0.

**C** A banking script calculates 0.10 + 0.20 and displays 0.30000000000000004 due to fractional binary conversion.

**D** A loop continues forever because the programmer wrote `count > 0` instead of `count < 0`.

ITEM 31

TOPIC: DEFINITION AND CAUSE OF ROUND-OFF ERRORS ★★★★★ VERY IMPORTANT

**Which of the following best explains why floating-point round-off errors occur in digital computers?**

**A** The program's source code contains grammatical spelling errors that the compiler fails to catch.

**B** Lossy compression algorithms intentionally delete color gradients from text documents.

**C** Real fractional numbers have infinite precision or non-terminating binary expansions, but computer memory allocates only a finite number of bits to store them.

**D** Integer variables exceed the maximum value representable by a 64-bit register, wrapping around to negative numbers.

ITEM 32

TOPIC: WHY 0.1 REPEATS IN BINARY REPRESENTATION ★★★★★ VERY IMPORTANT

**In base-10, the fraction 1/10 is represented cleanly as the terminating decimal 0.1. However, in binary (base-2), 1/10 is an infinitely repeating fraction: $0.0001100110011\ldots_2$. Why does this occur?**

**A** The ASCII character standard only encodes whole numbers, forcing all decimal fractions to repeat.

**B** Computer memory is organized into 8-bit bytes, and 8 cannot be divided by 10 without a remainder.

**C** A base-10 terminating fraction can only terminate in binary if its denominator's prime factors are exclusively powers of 2; because 10 has a prime factor of 5, 1/10 cannot terminate in base-2.

**D** Binary computers are not capable of performing division operations on numbers less than 1.

ITEM 33

TOPIC: FLOATING-POINT EQUALITY COMPARISON PITFALL ★★★★★ VERY IMPORTANT

**A student writes the following conditional check in a programming language:**

```
IF (0.1 + 0.2 == 0.3)
{
    DISPLAY("EQUAL")
}
ELSE
{
    DISPLAY("NOT EQUAL")
}
```

**When executed, the program unexpectedly displays 'NOT EQUAL'. What explains this outcome?**

**A** Due to floating-point round-off error, 0.1 + 0.2 evaluates to approximately 0.30000000000000004 in binary floating point, which is not strictly equal to the binary approximation of 0.3.

**B** The program crashed because computers cannot evaluate decimal points inside conditional statements.

**C** An integer overflow error occurred during the addition, rolling the sum over to 0.

**D** The `+` operator automatically concatenates numbers as text strings, producing '0.10.2' which does not equal '0.3'.

ITEM 34

TOPIC: ACCUMULATION OF ROUND-OFF ERROR IN REPETITIVE LOOPS

★★★★ HIGH PRIORITY

**A simulation program executes a loop that adds 0.001 to a floating-point accumulator variable one million times (intended to reach exactly 1,000.0). At the end of the simulation, the accumulator stores 999.999824. Which of the following best describes what occurred?**

**A** The CPU experienced thermal throttling, which dropped the least significant bit of the accumulator.

**B** A lossy compression filter removed decimal precision to save RAM during loop execution.

**C** The program suffered an integer overflow when the loop counter reached one million.

**D** Small floating-point round-off errors in each individual addition step accumulated over one million iterations, compounding into a measurable deviation.

ITEM 35

TOPIC: 1991 PATRIOT MISSILE SYSTEM ROUND-OFF CATASTROPHE

★★★★★ VERY IMPORTANT

**In February 1991, an air defense missile battery in Dhahran, Saudi Arabia failed to intercept an incoming ballistic missile, resulting in 28 fatalities. The defense system calculated time in tenths of a second using a 24-bit register. The value 1/10 was truncated to 24 bits, introducing a round-off error of 0.000000095 seconds per tick. After running continuously for 100 hours, what happened?**

**A** The radar communication system suffered a syntax error when receiving target coordinates.

**B** The round-off error accumulated to approximately 0.34 seconds; at supersonic speeds, this caused the tracking system to miscalculate the missile's position by over 500 meters.

**C** The missile's rocket engine ran out of fuel because an overflow error set the fuel gauge to zero.

**D** The system lost power because 100 hours exceeded the maximum battery capacity.

ITEM 36

TOPIC: SAFE FLOATING-POINT COMPARISON WITH EPSILON

★★★★ HIGH PRIORITY

**Which of the following techniques is considered the industry best practice when comparing two floating-point variables `a` and `b` for equality in mission-critical software?**

**A** Always use the strict equality operator `a == b` because modern compilers automatically eliminate all rounding errors.

**B** Convert both floating-point numbers to 8-bit unsigned integers before checking equality.

**C** Check whether the absolute difference between `a` and `b` is less than a small predefined threshold (epsilon): `abs(a - b) < epsilon`.

**D** Round both numbers to the nearest whole integer and compare their rounded integer values.

ITEM 37

TOPIC: FIXED-POINT / INTEGER-CENTS FINANCIAL STRATEGY

★★★★ HIGH PRIORITY

**An e-commerce payment platform processes transactions in US dollars. To avoid floating-point round-off errors that could cause fractions of a cent to vanish or accumulate illegally, the engineering team decides to store all prices as whole integers representing cents (e.g., $19.99 is stored as the integer 1999). What is the primary advantage of this design?**

**A** Integer arithmetic is completely exact and avoids floating-point round-off errors, ensuring accurate penny-perfect accounting.

**B** Storing prices as cents automatically prevents the database from ever experiencing integer overflow.

**C** Integer cents enable the website to accept payments in foreign currencies without currency exchange calculations.

**D** Storing integers allows prices to be compressed with lossy algorithms to reduce database disk size.

ITEM 38

TOPIC: FLOATING-POINT ARCHITECTURE (SIGN, EXPONENT, MANTISSA)

★★★ MANDATORY

**Under the standard IEEE 754 floating-point format, a real number is represented in scientific binary notation using three distinct components: sign bit, exponent, and mantissa (significand). What trade-off is made when using floating-point numbers compared to fixed-width integers?**

**A** Floating-point numbers can represent an immense range of both extremely large and microscopically small values, but at the cost of potential precision loss and round-off errors.

**B** Floating-point numbers eliminate all overflow errors, but can only store positive numbers.

**C** Floating-point numbers use fewer physical transistors in hardware, but require internet access to compute square roots.

**D** Floating-point numbers guarantee absolute mathematical precision with infinite decimal places, but are limited to numbers under 100.

ITEM 39

TOPIC: TRUNCATION VS. ROUNDING

★★★ MANDATORY

**When a floating-point calculation produces a binary value with more bits than the register's mantissa can hold, the system must drop the extra lower-order bits. If the hardware uses **truncation** (chopping) instead of rounding to the nearest value, what systemic effect occurs across many calculations?**

**A** The system triggers an arithmetic overflow exception on every subtraction operation.

**B** Calculations develop a systematic downward bias because positive numbers are always rounded down toward zero.

**C** All negative numbers are converted into positive numbers.

**D** Calculations automatically cancel out all rounding errors, producing exact mathematical results.

ITEM 40

TOPIC: COMPARISON: INTEGER OVERFLOW VS. FLOATING-POINT ROUND-OFF

★★★★★ VERY IMPORTANT

**Consider the following summary table comparing two core numerical limitations in computer science:**

| Attribute | Phenomenon X | Phenomenon Y |
|---|---|---|
| **Primary Cause** | Number is too large for the allocated integer bits | Real fractional number cannot be represented exactly in binary bits |
| **Typical Symptom** | Value wraps around (e.g., 255 becomes 0) | Small precision discrepancy (e.g., 0.1 + 0.2 = 0.30000000000000004) |

**Which of the following correctly identifies Phenomenon X and Phenomenon Y?**

**A** Phenomenon X is Lossless Compression; Phenomenon Y is Lossy Compression.

**B** Phenomenon X is Syntax Error; Phenomenon Y is Runtime Exception.

**C** Phenomenon X is Floating-Point Round-Off; Phenomenon Y is Integer Overflow.

**D** Phenomenon X is Integer Overflow; Phenomenon Y is Floating-Point Round-Off.

ITEM 41

TOPIC: CONTINUOUS ANALOG VS. DISCRETE DIGITAL DATA

★★★★★ VERY IMPORTANT

**Which of the following best describes the fundamental difference between continuous (analog) data and discrete (digital) data?**

**A** Analog data cannot be heard by human ears, whereas digital data is easily audible.

**B** Analog data is stored exclusively on magnetic hard drives, whereas digital data is stored exclusively in RAM.

**C** Analog data changes smoothly and continuously with an infinite number of possible intermediate states, whereas digital data is broken into discrete, separate values represented by binary bits.

**D** Analog data is completely immune to physical noise, whereas digital data degrades with every copy.

ITEM 42

TOPIC: THE DIGITIZATION SAMPLING PROCESS

★★★★★ VERY IMPORTANT

**A musician records a live acoustic guitar performance into audio recording software. Which of the following describes how the microphone and computer convert the analog sound wave into digital data?**

**A** The analog audio wave is measured (sampled) at regular time intervals, and each measured voltage is converted into a binary number.

**B** The computer stores the continuous physical vibration of the guitar strings directly onto the solid-state drive.

**C** The microphone compresses the acoustic wave using lossy JPEG compression before sending it to the sound card.

**D** The computer measures the sound wave once at the beginning of the song and estimates the remainder of the performance using an algorithm.

ITEM 43

TOPIC: IMPACT OF SAMPLING RATE ON AUDIO QUALITY

★★★★★ VERY IMPORTANT

A podcast producer tests two different recording settings for an interview:

- Setting A: 11,025 samples per second (11.025 kHz)
- Setting B: 44,100 samples per second (44.1 kHz)

Assuming the bit depth remains constant at 16 bits in both settings, which of the following correctly compares Setting B to Setting A?

**A** Setting B captures higher audio frequencies and reproduces sound with much higher fidelity, but produces a file that is four times larger.

**B** Setting B produces a file that is four times smaller, but introduces significant background static.

**C** Setting B introduces quantization errors that convert vocal recordings into instrumental music.

**D** Setting B doubles the dynamic volume range without altering the file size.

ITEM 44

TOPIC: BIT DEPTH (RESOLUTION) AND QUANTIZATION LEVELS

★★★★★ VERY IMPORTANT

In digital audio recording, what does **bit depth** represent, and how does increasing the bit depth from 8 bits to 16 bits affect the recording?

**A** Bit depth determines the number of audio channels (e.g., mono vs stereo) that can play simultaneously.

**B** Bit depth determines the number of bits allocated to store the amplitude of each sample; increasing from 8 to 16 bits increases the number of available amplitude levels from 256 to 65,536, dramatically reducing quantization noise.

**C** Bit depth determines the maximum length in minutes that an audio file can play.

**D** Bit depth determines how many times per second the sound wave is sampled; increasing to 16 bits doubles the recording speed.

ITEM 45

TOPIC: DIGITAL AUDIO FILE SIZE CALCULATION

★★★★ HIGH PRIORITY

A CD-quality stereo audio track is recorded using a sampling rate of 44,100 Hz, a bit depth of 16 bits per sample, and 2 audio channels (stereo). Which mathematical expression correctly calculates the total number of bits required to store 60 seconds of this uncompressed audio?

**A** (44,100 + 16 + 2) × 60 bits

**B** 44,100 × 16 × 2 × 60 bits

**C** 44,100 × 2 / 16 × 60 bits

**D** 44,100 × 16 × 60 / 2 bits

ITEM 46

TOPIC: QUANTIZATION ERROR DEFINITION

★★★★ HIGH PRIORITY

**When an analog sensor voltage of 3.4278 Volts is converted into digital representation, the system rounds it to the nearest discrete digital level of 3.4300 Volts. The discrepancy (+0.0022 V) is an example of which phenomenon?**

**A** Syntax compilation error

**B** Integer overflow error

**C** Lossless dictionary compression

**D** Quantization error (quantization noise)

ITEM 47

TOPIC: IMAGE DIGITIZATION (SPATIAL SAMPLING & COLOR DEPTH)

★★★★ HIGH PRIORITY

**A flatbed scanner digitizes an oil painting into an image file. How does the scanner apply the sampling and quantization process to visual images?**

**A** The continuous 2D painting is divided into a discrete grid of tiny picture elements (pixels), and each pixel's color is measured and quantized into binary values (such as 24-bit RGB).

**B** The scanner translates the artist's brush strokes into mathematical English sentences stored in an ASCII text file.

**C** The scanner converts the painting into a continuous audio waveform stored on a vinyl disc.

**D** The scanner counts the total weight of paint pigment and assigns an integer hash code.

ITEM 48

TOPIC: TRADE-OFFS IN SAMPLING PRECISION AND STORAGE

★★★★★ VERY IMPORTANT

**A mobile wildlife camera records continuous ambient bird songs in a national park. The rangers need the audio files to be transmitted wirelessly over a low-bandwidth cellular link while preserving enough clarity to identify species. Which of the following describes the key trade-off the rangers must balance when configuring the recording parameters?**

**A** Higher sampling rates produce smaller files that transmit faster, but introduce severe integer overflow errors.

**B** Higher sampling rates and bit depths capture greater acoustic detail for accurate bird identification, but produce larger files that take longer to transmit and consume more storage.

**C** Lower sampling rates capture higher audio frequencies, but require more expensive cellular antennas.

**D** Digital recordings can achieve infinite acoustic precision without using any storage space if lossy compression is enabled.

ITEM 49

TOPIC: DIGITAL EXACT REPRODUCTION VS. ANALOG GENERATION LOSS

★★★ MANDATORY

**When an audio cassette tape is copied from one physical tape to another (analog-to-analog), the copy sounds slightly degraded with noticeable background hiss. However, when an MP3 or WAV file is copied from a laptop to a flash drive (digital-to-digital), the copy sounds completely identical to the original. Why do digital copies avoid generation loss?**

**A** Digital files can only be copied once, after which operating systems permanently lock the file.

**B** Digital copies simply duplicate exact discrete binary bit values (0s and 1s), preventing physical signal noise from corrupting the underlying data.

**C** Digital flash drives operate at sub-zero temperatures that eliminate magnetic interference.

**D** Digital audio uses analog vacuum tubes that filter out tape hiss during file copying.

ITEM 50

TOPIC: CLASSIFYING REAL-WORLD SYSTEMS AS ANALOG OR DIGITAL

★★★★ HIGH PRIORITY

**Consider the following four measurement instruments:**

**1. A traditional mercury thermometer showing fluid level rising smoothly along a glass tube.**
**2. A digital kitchen scale displaying weight as discrete numbers on an LCD screen.**
**3. A mechanical vinyl record with microscopic grooves vibrating a physical needle.**
**4. An e-book reader displaying text as black and white pixels.**

**Which of the following correctly classifies these four items?**

**A** Items 1 and 2 are analog; Items 3 and 4 are digital.

**B** All four items are digital because they all measure physical quantities.

**C** Items 1 and 3 are analog; Items 2 and 4 are digital.

**D** Items 2 and 3 are analog; Items 1 and 4 are digital.

ITEM 51

TOPIC: DEFINITION OF LOSSLESS COMPRESSION

★★★★★ VERY IMPORTANT

**Which of the following best defines **lossless compression** in computer science?**

**A** A compression algorithm that converts binary data into analog sound waves to save hard drive space.

**B** A compression algorithm that only functions on encrypted text files.

**C** A compression algorithm that reduces file size while allowing the exact original data to be perfectly reconstructed bit-for-bit upon decompression.

**D** A compression algorithm that permanently discards imperceptible data to achieve maximum file size reduction.

ITEM 52

TOPIC: SCENARIOS REQUIRING LOSSLESS COMPRESSION

★★★★★ VERY IMPORTANT

For which of the following file types is **lossless compression** strictly required rather than lossy compression?

**A** Background music played in a crowded restaurant.

**B** A streaming video broadcast watched on a smartphone over a mobile cellular network.

**C** A casual selfie photo taken for a social media status update.

**D** A compiled software executable (.exe) or computer source code file, where losing or altering a single bit would corrupt program execution.

ITEM 53

TOPIC: RUN-LENGTH ENCODING (RLE) MECHANICS

★★★★★ VERY IMPORTANT

A monochrome bitmap image row contains the following 16 consecutive pixels, where 'W' represents white and 'B' represents black:

`WWWWWWWWBBBWWWWW`

How is this sequence represented using standard Run-Length Encoding (RLE)?

**A** `8W 5B 3W`

**B** `W8 B3 W5`

**C** `16W B`

**D** `8W 3B 5W`

ITEM 54

TOPIC: LIMITATIONS OF RUN-LENGTH ENCODING (RLE)

★★★★ HIGH PRIORITY

Under which of the following conditions will Run-Length Encoding (RLE) actually **increase** the file size rather than compress it?

**A** When a document consists entirely of the same repeated letter for 1,000 characters.

**B** When the data consists of rapidly alternating individual symbols with no consecutive repetitions (e.g., `WBWBWBWBWB`).

**C** When an image has large solid blocks of identical background color.

**D** When the original file is already encoded in hexadecimal.

ITEM 55

TOPIC: DICTIONARY-BASED LOSSLESS COMPRESSION MECHANICS

★★★★★ VERY IMPORTANT

A text compression tool processes a children's story containing repeated phrases. The tool constructs the following dictionary:

- Token 1 = "the little bear"
- Token 2 = "in the forest"
- Token 3 = "looked for berries"

The original sentence is:
*"The little bear in the forest looked for berries, and the little bear looked for berries again."*

Which of the following best describes how dictionary compression reduces file size?

**A** It converts text into low-resolution JPEG images to save characters.

**B** It permanently deletes all vowels from the text to make words shorter.

**C** It replaces lengthy, frequently repeated patterns of text with short numeric tokens, storing the dictionary alongside the encoded tokens.

**D** It replaces all English words with random 8-bit ASCII punctuation symbols.

ITEM 56

TOPIC: DECODING DICTIONARY-COMPRESSED DATA

★★★★ HIGH PRIORITY

Consider the following dictionary and encoded sequence:

**Dictionary:**
- 1: "blue"
- 2: "sky"
- 3: "clear"
- 4: "the"

**Encoded Stream:**
`4 1 2 4 3 2`

What is the exact decompressed text string produced by this data?

**A** "clear blue sky the sky"

**B** "blue sky the clear sky the"

**C** "the blue sky the clear sky"

**D** "the sky blue clear sky the"

ITEM 57

TOPIC: COMPRESSION RATIO AND PERCENTAGE SAVINGS CALCULATION

★★★★ HIGH PRIORITY

A high-resolution scientific data log originally occupies 400 Megabytes (MB). After applying a lossless ZIP compression algorithm, the resulting archive occupies 100 MB. What is the compression ratio and the percentage reduction in storage space?

**A** Compression ratio of 4:1 (75% space reduction)

**B** Compression ratio of 1:4 (400% space reduction)

**C** Compression ratio of 4:1 (25% space reduction)

**D** Compression ratio of 2:1 (50% space reduction)

ITEM 58

TOPIC: COMMON LOSSLESS IMAGE AND AUDIO FORMATS ★★★ MANDATORY

**Which of the following lists contains **only** file formats that utilize lossless compression?**

**A** JPEG, MP3, and MP4

**B** MP4, FLAC, and GIF

**C** JPEG, PNG, and MP3

**D** PNG, GIF, and FLAC

ITEM 59

TOPIC: THEORETICAL LIMITS OF LOSSLESS COMPRESSION (PIGEONHOLE PRINCIPLE) ★★★★ HIGH PRIORITY

**A software startup claims to have invented a universal lossless compression algorithm that guarantees to reduce the file size of **every single possible file**, no matter what data the file contains. Why is this claim mathematically impossible according to computer science theory?**

**A** Hardware memory chips physically reject files smaller than 1 kilobyte.

**B** Lossless compression algorithms are legally banned from compressing encrypted files.

**C** Binary computers can only perform compression on prime numbers.

**D** By the Pigeonhole Principle, there are fewer short bit-sequences than long bit-sequences; if every file were compressed to a shorter length, multiple distinct original files would map to the identical compressed sequence, making 100% lossless decompression impossible.

ITEM 60

TOPIC: COMPARING RLE VS. DICTIONARY-BASED COMPRESSION ★★★★ HIGH PRIORITY

**A software architect evaluates two lossless compression methods for an application:**

- **Algorithm 1: Run-Length Encoding (RLE)**
- **Algorithm 2: Dictionary-Based Compression (LZW)**

**Which of the following best describes when Algorithm 1 should be preferred over Algorithm 2?**

**A** Algorithm 1 (RLE) discards imperceptible data, while Algorithm 2 preserves all bits.

**B** Algorithm 1 (RLE) is preferred for streaming 4K video, while Algorithm 2 is used exclusively for printer drivers.

**C** Algorithm 1 (RLE) is preferred when the dataset contains long contiguous runs of identical repeated values (such as simple icon graphics with solid backgrounds), whereas Algorithm 2 is superior for natural language text with repeated words separated by other characters.

**D** Algorithm 1 (RLE) can compress random noise, while Algorithm 2 cannot.

ITEM 61

TOPIC: DEFINITION AND CORE MECHANISM OF LOSSY COMPRESSION

★★★★★ VERY IMPORTANT

**Which of the following best defines **lossy compression** in digital media processing?**

**A** A hardware malfunction that corrupts memory chips during file downloads.

**B** A compression process that temporarily hides data in a secondary file and restores all original bits when opened.

**C** A compression process that achieves significant file size reduction by permanently discarding data that is deemed redundant or imperceptible to human senses, making the original data impossible to reconstruct bit-for-bit.

**D** A compression process that only functions on text documents written in English.

ITEM 62

TOPIC: PERCEPTUAL ENCODING PRINCIPLES (PSYCHOACOUSTICS & PSYCHOVISUALS)

★★★★★ VERY IMPORTANT

**Lossy audio algorithms (such as MP3) and image algorithms (such as JPEG) achieve massive file size reductions without appearing grossly distorted to users. What biological principle makes this possible?**

**A** Human brains automatically manufacture extra bits of RAM when looking at digital smartphone screens.

**B** Human sensory organs have biological thresholds: the human eye is far more sensitive to changes in brightness (luminance) than subtle color shifts (chrominance), and human ears cannot perceive extremely high frequencies or sounds drowned out by louder nearby frequencies.

**C** All humans have identical eyesight and hearing that can be calibrated with a single universal binary formula.

**D** Lossy algorithms accelerate the physical speed of light emitted by LCD pixels to make images look sharper.

ITEM 63

TOPIC: IRREVERSIBILITY OF LOSSY COMPRESSION

★★★★★ VERY IMPORTANT

**A photographer takes a RAW uncompressed digital photo (50 MB) and exports it as a compressed JPEG photo (3 MB). The photographer then accidentally deletes the original 50 MB RAW file from their hard drive. Can the photographer convert the 3 MB JPEG back to the original 50 MB RAW file?**

**A** Yes, provided the photographer renames the file extension from `.jpg` to `.raw` in the file explorer.

**B** No, because lossy compression permanently discards image data; once removed, the original bits cannot be recovered or reconstructed.

**C** Yes, because computers save a hidden analog backup copy of every pixel on the motherboard.

**D** Yes, because all image editors can reverse JPEG compression using standard unzipping software.

ITEM 64

TOPIC: LOSSY COMPRESSION ARTIFACTS

★★★★ HIGH PRIORITY

**When an image is compressed with an excessively aggressive lossy JPEG setting (e.g., 98% compression), strange visual distortions appear, such as visible 8×8 square grid blocks and blurry halos around high-contrast edges. What are these distortions called in computer science?**

**A** Syntax parsing errors

**B** Quantization errors in continuous analog waves

**C** Integer overflow glitches

**D** Compression artifacts

ITEM 65

TOPIC: FORMAT SELECTION: PNG VS. JPEG FOR LOGOS AND TEXT

★★★★★ VERY IMPORTANT

**A graphic designer creates a corporate website banner featuring crisp black text on a pure transparent background and a sharp vector-style company logo. Which image format should the designer choose, and why?**

**A** MP3, because audio formats prevent unauthorized users from screenshotting the company logo.

**B** JPEG, because its lossy algorithm compresses text better than images and eliminates all pixel borders.

**C** GIF, because it is the only format supported by modern smartphone touchscreens.

**D** PNG, because its lossless compression preserves crisp, sharp edges without ringing artifacts and natively supports transparent alpha channels.

ITEM 66

TOPIC: GENERATION LOSS THROUGH REPEATED LOSSY SAVING

★★★★ HIGH PRIORITY

**A meme creator opens a JPEG image, adds a caption, saves it as a JPEG, sends it to a friend who crops it, saves it as a JPEG, and posts it to a forum where it is re-compressed again. With each save cycle, the image quality noticeably deteriorates. What causes this phenomenon?**

**A** The image resolution shrinks by 50% automatically every time a file is emailed.

**B** Floating-point round-off errors gradually erase the file's metadata tags.

**C** Generation loss: each time a file is re-encoded with lossy compression, the algorithm discards another layer of data and amplifies existing compression artifacts.

**D** The RGB color palette runs out of binary bits and reverts to monochrome.

ITEM 67

TOPIC: COMPARISON OF COMPRESSION RATIOS: LOSSLESS VS. LOSSY

★★★★ HIGH PRIORITY

**A media streaming platform compares storage savings across its digital catalog. On average, what compression ratios are typically achievable with lossless algorithms compared to lossy algorithms?**

**A** Lossless algorithms achieve 100:1 ratios, while lossy algorithms can only achieve 1.1:1 ratios.

**B** Lossless algorithms typically achieve modest ratios of around 2:1 to 3:1 (50% to 66% reduction), whereas lossy algorithms can routinely achieve 10:1 to 20:1 or greater (90% to 95% reduction).

**C** Lossless algorithms produce files that are always larger than the uncompressed original.

**D** Both lossless and lossy algorithms achieve identical compression ratios of exactly 4:1 on all files.

ITEM 68

TOPIC: IDENTIFYING COMMON LOSSY MULTIMEDIA FORMATS

★★★ MANDATORY

**Which of the following groups of digital file formats consists **entirely** of formats that employ lossy compression?**

**A** PNG, MP3, and FLAC

**B** JPEG, MP3, and MP4

**C** PNG, FLAC, and ZIP

**D** BMP, WAV, and TIFF

ITEM 69

TOPIC: EVALUATING SUITABILITY OF LOSSY COMPRESSION IN HEALTHCARE

★★★★★ VERY IMPORTANT

**A hospital radiological imaging department archives high-resolution digital X-rays and MRI scans used by oncologists to detect micro-calcifications and early-stage tumors. Why does hospital policy **prohibit** the use of lossy compression for diagnostic patient scans?**

**A** Lossy compression permanently discards fine detail and introduces subtle pixel artifacts that could obscure an early-stage tumor or simulate a false micro-fracture, leading to fatal misdiagnosis.

**B** Lossy compression files can only be opened on mobile smartphones, not hospital computer monitors.

**C** Lossy compression algorithms delete patient legal names and insurance billing records from hospital databases.

**D** Hospital hard drives are physically incapable of storing JPEG files due to electrical interference from X-ray machines.

ITEM 70

TOPIC: COMPREHENSIVE COMPRESSION DECISION MATRIX

★★★★ HIGH PRIORITY

**Consider the following summary table of four project requirements:**

| Project | Primary Constraint | Requirement |
|---|---|---|
| 1. Mobile Gaming App Textures | Cellular download cap under 50 MB | Low bandwidth and small app install size |
| 2. State Banking Tax Database | Regulatory audit compliance | Exact dollar balances must be preserved bit-for-bit |
| 3. High-Speed Satellite Audio Feed | Voice clarity needed over narrow radio link | Minimizing transmission delay and data rate |
| 4. Software Operating System Patch | System stability | Binary code must execute with zero altered bits |

**Which of the following correctly pairs each project with its appropriate compression strategy?**

**A** Projects 2 and 3 should use Lossy; Projects 1 and 4 must use Lossless.

**B** Projects 1 and 2 should use Lossy; Projects 3 and 4 must use Lossless.

**C** All four projects should use Lossless compression because storage is always free.

**D** Projects 1 and 3 should use Lossy; Projects 2 and 4 must use Lossless.

ITEM 71

TOPIC: DEFINITION AND ROLE OF METADATA

★★★★★ VERY IMPORTANT

**Which of the following best defines **metadata** in computer science?**

**A** A specialized form of computer virus that hides inside database tables.

**B** The physical electrical voltage flowing through an Ethernet cable.

**C** The compiled machine code instructions executed directly by the CPU.

**D** Data that provides descriptive information about other data, such as creation timestamps, author identity, file size, or resolution.

ITEM 72

TOPIC: EFFECT OF MODIFYING METADATA ON PRIMARY CONTENT

★★★★★ VERY IMPORTANT

**A user modifies the title tag, copyright note, and creation date in the file properties of a digital photograph, but does not open the image in a photo editor. How does this modification affect the actual image content (the pixels) of the photograph?**

**A** The image resolution is automatically cut in half whenever metadata is altered.

**B** The visual pixel colors and resolution of the photograph remain completely unchanged because metadata is stored separately from the primary image data.

**C** The photograph undergoes lossy compression, permanently blurring the center pixels.

**D** The photograph is converted into an uncompressed audio file.

ITEM 73

TOPIC: UTILITY OF METADATA IN SEARCH, INDEXING, AND ORGANIZATION

★★★★ HIGH PRIORITY

**A digital music streaming service manages a catalog of over 80 million songs. When a user searches for all 'Jazz' songs recorded in '1959', the application delivers results in less than 0.1 seconds. How does metadata enable this rapid search?**

**A** The service requires every musician to type their songs directly into SQL database tables in binary.

**B** The database indexes structured metadata fields (such as genre tags and release year) without needing to decompress or analyze the audio waveforms of the songs.

**C** The service plays all 80 million songs simultaneously in the background at 1,000x speed.

**D** The database uses lossy compression to delete every song that was not recorded in 1959.

ITEM 74

TOPIC: SMARTPHONE PHOTO EXIF METADATA COMPONENTS

★★★★★ VERY IMPORTANT

```
When a student snaps a photo using a modern smartphone camera, the operating system automatically embeds an EXIF (Exchan
```

**A** Date and time taken, GPS latitude and longitude coordinates, camera device model, and exposure/ISO settings.

**B** A complete video recording of everything that happened 24 hours prior to the photo.

**C** The student's banking password, social security number, and web browser history.

**D** The chemical composition of the phone's lithium-ion battery.

ITEM 75

TOPIC: PRIVACY AND SECURITY RISKS OF METADATA

★★★★★ VERY IMPORTANT

**A student takes a photograph of their new puppy inside their living room and posts the raw image file to a public online discussion board. An anonymous user successfully identifies the student's exact home street address. How was the anonymous user able to discover this private information?**

**A** The stranger converted the image into 7-bit ASCII and read the student's thoughts.

**B** The camera's CMOS sensor reflected the street sign in the puppy's eyes in 100% lossless resolution.

**C** The student failed to strip the EXIF metadata from the photo, allowing the stranger to read the embedded GPS latitude and longitude coordinates.

**D** The image was compressed with lossy compression, which converted the puppy's fur into street address letters.

ITEM 76

TOPIC: METADATA IN NETWORK COMMUNICATIONS (PACKET HEADERS) ★★★★ HIGH PRIORITY

**When an email message travels across the Internet, it is broken down into digital packets. In each packet, what is the relationship between the **payload** and the **metadata**?**

**A** The metadata is the body of the email, while the payload is the email subject line.

**B** The payload is discarded by routers to make packets travel faster, leaving only metadata.

**C** The payload is stored in hexadecimal, while metadata is stored exclusively in decimal.

**D** The payload is the actual content of the email text being delivered, while the metadata (packet header) contains routing information like source IP address, destination IP address, and packet sequence numbers.

ITEM 77

TOPIC: METADATA SCRUBBING / SANITIZATION ★★★★ HIGH PRIORITY

**Before publishing a confidential report on a corporate website, an organization runs a software tool that removes the original author's username, internal network file paths, and tracked revision history from the PDF document. What is this cybersecurity practice called?**

**A** Integer overflow mitigation

**B** Digital-to-analog conversion

**C** Lossy perceptual sampling

**D** Metadata scrubbing (sanitization)

ITEM 78

TOPIC: CLASSIFYING DATA AS PRIMARY CONTENT VS. METADATA ★★★ MANDATORY

**Consider the following five elements associated with an email message:**

**1. The written paragraph: 'Please review the attached project proposal.'**
**2. The timestamp: 'Oct 12, 2026, 8:15 AM EDT'**
**3. The sender email address: 'alice@school.edu'**
**4. The recipient email address: 'bob@school.edu'**
**5. The total file size: '24.6 KB'**

**Which of these elements represents the **primary data (content)**, and which represent **metadata**?**

**A** Element 1 is the primary data; Elements 2, 3, 4, and 5 are metadata.

**B** All five elements are metadata because they all relate to an email.

**C** Elements 1, 2, and 5 are primary data; Elements 3 and 4 are metadata.

**D** Element 5 is the primary data; Elements 1, 2, 3, and 4 are metadata.

ITEM 79

TOPIC: BIG DATA INSIGHTS DERIVED EXCLUSIVELY FROM METADATA

★★★★★ VERY IMPORTANT

**A telecommunications company provides researchers with millions of phone call records containing only metadata: caller phone number, recipient phone number, call start time, duration, and cell tower location (with all actual voice audio conversations strictly excluded). What can researchers still uncover from this metadata alone?**

**A** Social networks, daily movement patterns, relationships, and organizational hierarchies can be mapped with high accuracy, even without hearing the contents of any conversation.

**B** Researchers can determine the blood pressure and pulse of the callers.

**C** Nothing, because without the voice audio recordings, metadata contains zero useful information.

**D** Researchers can reconstruct the exact word-for-word spoken dialogue using artificial intelligence.

ITEM 80

TOPIC: AUDIO FILE ID3 METADATA TAGS

★★★ MANDATORY

**An MP3 music file includes an ID3 metadata tag containing the song title, artist name, album name, year of release, and album cover thumbnail image. If a music enthusiast uses a tag editor to correct a misspelled artist name, which of the following occurs?**

**A** The MP3 file is automatically upgraded to uncompressed 24-bit studio FLAC.

**B** The entire song must be re-recorded in a sound studio to match the new artist name.

**C** The ID3 metadata header is updated with the new text string, while the acoustic audio waveform data remains completely unaffected.

**D** The audio tempo speeds up by 5% because the text string was modified.

ITEM 81

TOPIC: TRANSFORMING RAW DATA INTO INFORMATION

★★★★★ VERY IMPORTANT

**Which of the following best describes the relationship between **raw data** and **information** in computing?**

**A** Raw data is always biased, while information is guaranteed to be 100% objective and error-free.

**B** Raw data consists of unorganized, raw facts, measurements, or symbols; information is produced when that data is processed, structured, and contextualized to reveal patterns or answer questions.

**C** Raw data can only be viewed by computers, while information is viewed exclusively in printed books.

**D** Raw data is stored in lossy formats, while information is stored exclusively in lossless formats.

ITEM 82

TOPIC: THE DATA CLEANING (SCRUBBING) PROCESS ★★★★★ VERY IMPORTANT

**A municipal transit agency collects passenger feedback through a digital kiosk. The resulting survey dataset contains thousands of rows, with issues such as:**

- **Inconsistent date formats ('10/12/26', 'October 12, 2026', '2026-10-12')**
- **Duplicate submissions from users who clicked 'Submit' multiple times**
- **Impossible values (e.g., passenger age listed as -5 or 450)**
- **Incomplete records with blank feedback fields**

**What is the systematic process of identifying and correcting these errors before conducting data analysis called?**

**A** Floating-point round-off estimation

**B** Data cleaning (data cleansing / data scrubbing)

**C** Digital-to-analog sampling

**D** Lossless run-length encoding

ITEM 83

TOPIC: CORRELATION VS. CAUSATION IN DATA SCIENCE ★★★★★ VERY IMPORTANT

**A health data science team analyzes regional municipal records and discovers a strong positive statistical correlation: in months where community ice cream sales are highest, hospital emergency room visits for severe sunburns are also at their highest. What is the most scientifically sound conclusion to draw from this data?**

**A** The dataset has an integer overflow error that created a fictitious correlation.

**B** Suffering a sunburn causes patients to develop intense biological cravings for cold dairy products.

**C** Eating ice cream chemically causes human skin to become sensitive to solar radiation, directly producing sunburns.

**D** Ice cream sales and sunburn cases correlate strongly because both are influenced by a common confounding variable (hot, sunny summer weather); eating ice cream does not cause sunburns.

ITEM 84

TOPIC: SAMPLING BIAS AND LIMITATIONS OF DATASETS ★★★★★ VERY IMPORTANT

**A city council wants to determine whether residents support replacing a local public park with a new municipal parking structure. They post an online survey exclusively on the city government's social media webpage at 2:00 PM on a Tuesday. Over 80% of respondents vote in favor of the parking garage. Why might this dataset be fundamentally flawed?**

**A** Selection (sampling) bias: the survey sampled only individuals with internet access who follow city social media and were online during workday hours, excluding working residents, elderly citizens without social media, and park visitors who do not use the platform.

**B** An integer overflow in the vote counter converted 1,000 'No' votes into 1,000 'Yes' votes.

**C** Lossy compression deleted all the 'No' votes from the database.

**D** The survey collected metadata, which legally invalidates all online elections.

ITEM 85

TOPIC: THE 3 VS OF BIG DATA (VOLUME, VELOCITY, VARIETY) ★★★★ HIGH PRIORITY

**A global weather forecasting service receives millions of sensor readings every second from satellites, ocean buoys, commercial aircraft, and radar installations, in formats ranging from radar images and text reports to continuous temperature streams. Which concept describes the computing challenge of managing this massive scale of data?**

**A** Run-Length Encoding, which requires all weather satellites to send data in black-and-white pixels.

**B** Single-core processor execution, which requires all weather calculations to run sequentially on a single laptop.

**C** Big Data, characterized by immense Volume (size), high Velocity (rapid influx), and diverse Variety (heterogeneous formats).

**D** Lossless dictionary compression, which replaces all cloud formations with English words.

ITEM 86

TOPIC: PARALLEL AND DISTRIBUTED PROCESSING FOR BIG DATA ★★★★ HIGH PRIORITY

**A genetic research institute needs to analyze 50,000 sequenced human genomes, a computation that would take 12 years to complete on a single high-end desktop workstation. How can the institute complete this analysis in a matter of days?**

**A** Compress all DNA sequences using lossy MP3 compression to make the calculations run 1,000 times faster.

**B** Use parallel and distributed processing across a cluster of hundreds of interconnected cloud servers, dividing the dataset into chunks that are analyzed simultaneously.

**C** Convert the DNA files from binary to 7-bit ASCII so they fit in CPU cache memory.

**D** Round all genetic base pairs to the nearest integer to reduce floating-point overhead.

ITEM 87

TOPIC: OUTLIER DETECTION AND HANDLING ★★★★ HIGH PRIORITY

**A team of meteorologists analyzes daily temperature records from an automated Arctic weather station over the month of January. Most daily readings range between -25°C and -35°C. However, on January 14th, the sensor recorded a temperature of +999.0°C. Which of the following describes the most appropriate data cleaning response?**

**A** Delete the entire month of January from the database to hide the discrepancy.

**B** Identify the +999.0°C reading as an erroneous outlier caused by sensor hardware malfunction or default error code, and filter or flag it so it does not distort monthly average statistics.

**C** Assume that a volcanic eruption instantly occurred and publish an emergency climate bulletin without checking the sensor.

**D** Include +999.0°C in the monthly average calculation because all recorded data points must always be treated as 100% physically accurate.

ITEM 88

TOPIC: HANDLING MISSING DATA IN LARGE DATASETS ★★★ MANDATORY

**A hospital database contains 100,000 patient records. In 500 records, the patient's blood pressure field is completely blank (missing data). Which of the following describes a valid trade-off when deciding how to handle these missing records?**

**A** Discarding all 500 incomplete records ensures the remaining data is complete, but risks introducing bias if patients with missing blood pressure share common demographic or medical characteristics.

**B** Encrypting the missing fields with lossy compression automatically fills in the correct blood pressure values.

**C** Missing data has no mathematical effect on statistical calculations such as averages or standard deviations.

**D** Replacing all missing blood pressure values with 0 eliminates bias and preserves exact clinical accuracy.

ITEM 89

TOPIC: DATA AGGREGATION AND LOSS OF GRANULARITY ★★★★ HIGH PRIORITY

**A municipal government publishes an economic report showing that the average household income in a city increased by 15% over the past decade. However, independent neighborhood reporters discover that 80% of families in the city actually experienced a decline in purchasing power. How can data aggregation explain this apparent contradiction?**

**A** Independent reporters must have misread the data because mathematical averages never distort reality.

**B** Average household income calculations suffer from integer overflow whenever salaries exceed $1,000.

**C** The city government compressed the income records using lossy JPEG compression, which added numbers together randomly.

**D** Aggregating data by taking a single citywide average (mean) masks localized inequality, because a massive wealth increase among a few ultra-wealthy households skewed the overall average upward.

ITEM 90

TOPIC: HISTORICAL BIAS AND ALGORITHMIC FAIRNESS ★★★★★ VERY IMPORTANT

**A large enterprise company trains a machine learning algorithm to screen job resumes by analyzing historical hiring records from the past 20 years. When tested, the algorithm systematically rates female applicants lower for technical engineering roles, even when their qualifications, degrees, and work experience match male applicants. What is the root cause of this defect?**

**A** The software experienced a floating-point round-off error when converting resumes into PDF files.

**B** The algorithm ran out of 8-bit ASCII characters for female names.

**C** Historical bias in the training dataset: because past hiring practices favored male candidates, the algorithm learned and amplified historical human prejudice from the historical data.

**D** The server experienced a network timeout during resume uploads.

## Binary Sensor Representation & Integer Overflow in a Smart Greenhouse Monitor

★★★★★ CED STANDARD

9 Points Total

**Program Scenario & System Architecture:**

A student developer creates an automated IoT greenhouse climate control system called 'EcoGrow'. The system monitors ambient humidity and soil moisture using hardware sensors connected to an 8-bit microcontroller.

The microcontroller stores the relative humidity percentage using an unsigned 8-bit binary register (`humidityReg`).

The sensor polling algorithm executes every minute:

```
WHEN humiditySensorTimer FIRES:
    rawReading <- readAnalogSensorVoltage()
    scaledHumidity <- convertToPercentage(rawReading)  // Scaled from 0 to 100

    IF (scaledHumidity >= 0 AND scaledHumidity <= 100)
    {
        humidityReg <- scaledHumidity
        updateDashboardDisplay(humidityReg)
    }
    ELSE
    {
        logSensorMalfunction(scaledHumidity)
    }
```

During a hardware test, an engineer bypasses the calibration subroutine and manually loads the binary bit pattern `$11111111_2$` into an unconstrained 8-bit counter tracking raw humidity pulses, and increments it by 1.

**Prompt Requirements:**

**(a) [2 Points]:** Identify the decimal (base-10) value represented by the 8-bit binary pattern $11111111_2$ and show the mathematical calculation using positional place values.

**(b) [2 Points]:** Explain what occurs at the hardware and software register level when the system adds 1 to $11111111_2$ in an unsigned 8-bit register, identifying the specific error type.

**(c) [3 Points]:** Describe the real-world operational consequence if this overflow error occurred while the automated greenhouse irrigation system was actively controlling sprinkler valves.

**(d) [2 Points]:** Provide two concrete software engineering modifications the developer could implement in the system algorithm or data type declarations to completely prevent this error.

***Exam Strategy & Reminders:*** *Show complete mathematical conversion steps where calculations are required. Explicitly distinguish between continuous analog physical phenomena vs discrete digital numeric representations, and justify trade-offs between storage size and data fidelity.*

## Analog Audio Digitization, Sampling Rate & Bit Depth in a Voice Memo App

★★★★★ CED STANDARD

9 Points Total

**Program Scenario & System Architecture:**

A student software developer creates a mobile application called 'NoteVoice' that allows high school students to record spoken voice lectures and listen to them on mobile devices.

During development, the student tests two distinct audio recording configurations:

• Configuration Alpha: Sampling Rate = 8,000 Hz (8 kHz), Bit Depth = 8 bits per sample, Mono (1 channel)
• Configuration Beta: Sampling Rate = 44,100 Hz (44.1 kHz), Bit Depth = 16 bits per sample, Stereo (2 channels)

Both configurations record the same 10-minute (600 seconds) classroom lecture.

**Prompt Requirements:**

**(a) [2 Points]:** Explain the fundamental difference between the continuous analog sound wave produced by the human speaker and the discrete digital data stored by NoteVoice.

**(b) [3 Points]:** Calculate the exact uncompressed file size in bits for both Configuration Alpha and Configuration Beta for the 10-minute lecture. Show all work.

**(c) [2 Points]:** Describe the specific auditory quality differences a listener will experience when playing back Configuration Beta compared to Configuration Alpha, explaining the roles of both sampling rate and bit depth.

**(d) [2 Points]:** Evaluate which configuration is more appropriate for low-cost smartphones operating on limited mobile cellular data caps, justifying your recommendation with a trade-off analysis.

***Exam Strategy & Reminders:*** *Show complete mathematical conversion steps where calculations are required. Explicitly distinguish between continuous analog physical phenomena vs discrete digital numeric representations, and justify trade-offs between storage size and data fidelity.*

## Run-Length Encoding vs. Dictionary-Based Lossless Compression in a Hospital Archive

★★★★★ CED STANDARD

9 Points Total

**Program Scenario & System Architecture:**

A hospital digital records management system archives two distinct categories of files:

• Category 1: Digital dental X-ray scans (monochrome bitmap images containing large solid regions of pure black background and solid white teeth enamel).
• Category 2: Medical billing and patient diagnostic reports (text documents written in English with frequently repeated clinical phrases such as 'no acute cardiopulmonary disease' and 'patient tolerated procedure well').

The hospital engineering team evaluates two lossless compression algorithms: Run-Length Encoding (RLE) and Dictionary-Based Compression (LZW).

**Prompt Requirements:**

**(a) [2 Points]:** Define what makes an algorithm 'lossless' and explain why hospital regulatory policies mandate lossless compression for both diagnostic X-rays and patient reports.

**(b) [3 Points]:** Demonstrate how Run-Length Encoding compresses the following 20-character pixel row from an X-ray scan: `BBBBBBBBBBWWWWBBBBBB` (where B=black, W=white). State the compressed representation and calculate the percentage character reduction.

**(c) [2 Points]:** Explain why Run-Length Encoding performs exceptionally well on Category 1 (dental X-rays) but performs poorly on Category 2 (English diagnostic text).

**(d) [2 Points]:** Explain how Dictionary-Based Compression successfully compresses Category 2 text files without losing any clinical information.

***Exam Strategy & Reminders:*** *Show complete mathematical conversion steps where calculations are required. Explicitly distinguish between continuous analog physical phenomena vs discrete digital numeric representations, and justify trade-offs between storage size and data fidelity.*

## Lossy JPEG Compression & Quality Trade-Offs in an Online School Newspaper

★★★★★ CED STANDARD

9 Points Total

**Program Scenario & System Architecture:**

A student journalism editorial team runs an online school newspaper website. High-resolution digital photographs taken with DSLR cameras (average size: 25 MB per photo in uncompressed TIFF format) are causing the school's web server to run out of disk storage and causing website articles to load extremely slowly on mobile phones.

The editor-in-chief recommends converting all images to JPEG format with a quality compression setting of 75%.

After conversion, the average photo size drops from 25 MB to 1.2 MB.

**Prompt Requirements:**

**(a) [2 Points]:** Identify the type of compression utilized by the JPEG algorithm (lossless or lossy) and explain what happens to the original pixel data during this conversion.

**(b) [2 Points]:** Calculate the compression ratio achieved by this conversion (from 25 MB to 1.2 MB) and the percentage of storage space saved. Show your work.

**(c) [3 Points]:** Describe two specific psychovisual human perception principles that allow JPEG to discard substantial image data without noticeable degradation to casual website readers.

**(d) [2 Points]:** Identify one visual compression artifact that could appear if the editor accidentally set the quality level to 5% (extreme compression), and explain why the original 25 MB quality can never be restored from the 1.2 MB JPEG.

***Exam Strategy & Reminders:*** *Show complete mathematical conversion steps where calculations are required. Explicitly distinguish between continuous analog physical phenomena vs discrete digital numeric representations, and justify trade-offs between storage size and data fidelity.*

## EXIF Photo Metadata Extraction & Privacy Protection in a Student Photo Gallery

★★★★★ CED STANDARD

9 Points Total

**Program Scenario & System Architecture:**

A student photography club builds a mobile app where students can upload and showcase their original photos. When a photo is uploaded, the app automatically reads the file header.

An uploaded photo from a student contains the following metadata header:

```
{
    "FileName": "sunset_park.jpg",
    "DateTimeOriginal": "2026-10-12 17:42:15",
    "CameraModel": "iPhone 15 Pro",
    "LensFocalLength": "24mm",
    "ExposureTime": "1/250 sec",
    "ISO": 100,
    "GPSLatitude": "37.7749 N",
    "GPSLongitude": "122.4194 W",
    "FileSize": "3,412,890 bytes",
    "ColorSpace": "sRGB"
}
```

A student officer notices that photos taken inside students' private residences also contain exact GPS coordinates.

**Prompt Requirements:**

**(a) [2 Points]:** Define what metadata is and explain the distinction between the metadata shown above and the primary image data of the photo.

**(b) [2 Points]:** Identify two specific fields in the metadata above that pose severe personal privacy and security risks to students if published publicly on the school club website.

**(c) [2 Points]:** Explain how metadata benefits the photography club app for non-sensitive features, providing two legitimate functional use cases.

**(d) [3 Points]:** Describe the specific technical workflow the mobile app should execute during photo upload to sanitize (scrub) sensitive metadata while preserving image quality.

***Exam Strategy & Reminders:*** *Show complete mathematical conversion steps where calculations are required. Explicitly distinguish between continuous analog physical phenomena vs discrete digital numeric representations, and justify trade-offs between storage size and data fidelity.*

## Data Cleaning, Deduplication & Outlier Detection in a Municipal Health Survey

★★★★★ CED STANDARD

9 Points Total

**Program Scenario & System Architecture:**

A municipal public health agency collects 50,000 digital survey submissions from city residents regarding neighborhood exercise habits and sleep duration. The raw survey data table contains several data quality defects:

| RecordID | ResidentAge | DailySleepHours | ExerciseMinutes | SurveyDate |
|---|---|---|---|---|
| **10421** | 24 | 7.5 | 45 | 2026-10-12 |
| **10422** | -8 | 8.0 | 30 | 10/12/2026 |
| **10423** | 42 | 6.0 | 60 | 2026-10-12 |
| **10424** | 42 | 6.0 | 60 | 2026-10-12 |
| **10425** | 31 | 99.0 | 0 | Oct 12, 2026 |
| **10426** | 55 | [BLANK] | 20 | 2026-10-12 |

The agency's chief data analyst must clean this dataset prior to generating public health policy reports.

**Prompt Requirements:**

**(a) [3 Points]:** Identify three distinct data anomalies or defects present in the sample table above and classify each error type.

**(b) [2 Points]:** Explain the difference between Records 10423 and 10424, and describe how deduplication software handles this situation.

**(c) [2 Points]:** Analyze Record 10425 (`DailySleepHours = 99.0`). Explain whether this value represents a genuine observation or an erroneous outlier, and describe how leaving it uncorrected would skew analytical statistics.

**(d) [2 Points]:** Propose two distinct analytical strategies for handling Record 10426 with the blank `DailySleepHours` field, evaluating the trade-off of each approach.

***Exam Strategy & Reminders:*** *Show complete mathematical conversion steps where calculations are required. Explicitly distinguish between continuous analog physical phenomena vs discrete digital numeric representations, and justify trade-offs between storage size and data fidelity.*

## Floating-Point Round-Off Error & Financial Accumulation in a School Meal Account System

★★★★★ CED STANDARD

9 Points Total

**Program Scenario & System Architecture:**

A high school district manages student cafeteria meal accounts using an automated billing system. In the accounting software, student balances are stored as standard 64-bit binary floating-point numbers (`balanceFloat`).

A student deposits $10.00 into their account. Over the school week, the student purchases items costing $0.10 and $0.20.

The transaction script contains the following code segment:

```
itemPrice1 <- 0.10
itemPrice2 <- 0.20
totalCharged <- itemPrice1 + itemPrice2
```

```
IF (totalCharged == 0.30)
{
    applyStudentDiscount()
}
ELSE
{
    logDiscrepancyNotice()
}
```

Over the course of a school year with 500,000 transactions, the school district's financial ledger reports a mysterious $43.12 accounting imbalance.

**Prompt Requirements:**

**(a) [2 Points]:** Explain why `totalCharged` does not equal 0.30 in binary floating-point arithmetic, identifying why the ELSE branch executed.

**(b) [2 Points]:** Explain why decimal values like 0.10 and 0.20 cannot be represented with terminating precision in binary computer memory.

**(c) [2 Points]:** Describe how microscopic rounding errors in individual 10-cent and 20-cent transactions accumulated into a $43.12 discrepancy over 500,000 transactions.

**(d) [3 Points]:** Propose an alternative numerical data representation strategy that completely eliminates floating-point round-off errors in financial ledgers, demonstrating how the transaction script would be rewritten.

***Exam Strategy & Reminders:*** *Show complete mathematical conversion steps where calculations are required. Explicitly distinguish between continuous analog physical phenomena vs discrete digital numeric representations, and justify trade-offs between storage size and data fidelity.*

## Hexadecimal Color Representation & Bitwise RGB Channels in an Accessibility Screen Tint Tool

★★★★★ CED STANDARD

9 Points Total

**Program Scenario & System Architecture:**

A student software developer creates an operating system accessibility utility called 'VisionClear' that adjusts screen color palettes for users with color vision deficiency (color blindness).

The software allows users to enter custom 24-bit hexadecimal color codes formatted as `#RRGGBB` to tint the screen.

A user enters the hexadecimal color code `#9B30FF`.

The application interface provides three slider controls displaying the decimal intensity (0 to 255) for each color channel: Red, Green, and Blue.

**Prompt Requirements:**

**(a) [3 Points]:** Convert the hexadecimal color code `#9B30FF` into its three constituent decimal channel intensities for Red, Green, and Blue. Show all mathematical conversion steps.

**(b) [2 Points]:** Convert the Red channel intensity (`$9B_{16}$`) into its exact 8-bit binary representation.

**(c) [2 Points]:** Explain why computer displays use hexadecimal notation rather than binary to represent 24-bit color codes in software interfaces.

**(d) [2 Points]:** Describe what visual color will be displayed if the user changes the code to `#000000`, and explain how the additive RGB light model produces pure white (`#FFFFFF`).

***Exam Strategy & Reminders:*** *Show complete mathematical conversion steps where calculations are required. Explicitly distinguish between continuous analog physical phenomena vs discrete digital numeric representations, and justify trade-offs between storage size and data fidelity.*

## Big Data Parallel Processing & Correlation vs. Causation in an Academic Analytics Study

★★★★★ CED STANDARD

9 Points Total

**Program Scenario & System Architecture:**

A university data science laboratory collects 10 Terabytes of digital student activity data across 40,000 undergraduate students over four years, including:

- Wi-Fi campus building connection logs (timestamps and access point IDs)
- Digital learning management system (LMS) clickstreams and submission timestamps
- Dining hall card swipe logs
- Final semester grade point averages (GPAs)

An automated correlation analysis discovers a strong positive correlation ($r = 0.88$): students who connect to campus library Wi-Fi after 11:00 PM achieve higher GPAs than students who do not.

**Prompt Requirements:**

**(a) [3 Points]:** Identify the 3 Vs of Big Data (Volume, Velocity, Variety) and demonstrate how this university dataset exhibits each attribute.

**(b) [2 Points]:** Explain why analyzing 10 Terabytes of distributed log data requires parallel computing across a cluster of servers rather than sequential processing on a single computer.

**(c) [2 Points]:** Analyze the positive correlation between late-night library Wi-Fi connections and high GPAs. Explain why concluding that 'forcing students to connect to library Wi-Fi after 11:00 PM will raise their grades' is an analytical fallacy, identifying a likely confounding variable.

**(d) [2 Points]:** Identify two serious privacy and surveillance risks associated with collecting continuous Wi-Fi location logs on university students.

***Exam Strategy & Reminders:*** *Show complete mathematical conversion steps where calculations are required. Explicitly distinguish between continuous analog physical phenomena vs discrete digital numeric representations, and justify trade-offs between storage size and data fidelity.*

## Comprehensive Data Pipeline: From Analog Environmental Sampling to Lossless Cloud Telemetry

★★★★★ CED STANDARD

9 Points Total

**Program Scenario & System Architecture:**

A team of environmental engineering students deploys an autonomous solar-powered weather buoy in a marine sanctuary. The buoy measures ocean water temperature continuously.

The system data pipeline consists of four sequential stages:

```
1. **Stage 1 (Sensor Sampling):** An analog thermistor generates a continuous voltage signal. An Analog-to-Digital Converte
2. **Stage 2 (Calibration & Clamping):** The reading is converted into a temperature reading between 0.0°C and 40.0°C.
3. **Stage 3 (Data Compression):** Consecutive identical temperature readings during calm night cycles are compressed using
4. **Stage 4 (Satellite Transmission):** The compressed temperature payload is packaged into a digital telemetry packet wit
```

**Prompt Requirements:**

**(a) [2 Points]:** Explain how Stage 1 converts the continuous analog ocean temperature into discrete digital data, and calculate the total number of distinct quantization levels provided by the 10-bit ADC.

**(b) [2 Points]:** During a calm night, the buoy records the following 12 temperature readings (in °C): `[18, 18, 18, 18, 18, 18, 19, 19, 19, 18, 18, 18]`. Show the exact Run-Length Encoded representation of this dataset.

**(c) [2 Points]:** Explain why the engineering team chose lossless Run-Length Encoding rather than lossy JPEG/MP3-style compression for environmental ocean telemetry.

**(d) [3 Points]:** In Stage 4, distinguish between the packet's metadata header and its primary data payload, and explain how a satellite station utilizes both components upon reception.

***Exam Strategy & Reminders:*** *Show complete mathematical conversion steps where calculations are required. Explicitly distinguish between continuous analog physical phenomena vs discrete digital numeric representations, and justify trade-offs between storage size and data fidelity.*

# End of Question Paper

You have reached the end of Questions 1 through 100.

> **Before you continue:** Make sure you have genuinely attempted every question under timed examination conditions. The pages that follow contain the complete multiple choice master answer key, fully worked step-by-step solutions with detailed distractor failure analysis, and model Create Performance Task written responses with official 9-point AP scoring rubrics.

## What Comes Next

❖ **Answer Key (Page 44):** Letter answers for Questions 1–90 with priority star ratings.

❖ **Multiple Choice Solutions (Pages 45–74):** Step-by-step reasoning for Questions 1–90 plus an explanation of why each incorrect option fails.

❖ **Free Response Solutions (Pages 75–84):** Complete model answers for Questions 91–100 with point-by-point 9-point rubrics.

❖ **Appendix & Reference Sheet (Page 85):** Essential Unit 2 Data Representation Framework, Powers of 2 Table, Sampling Formulas & Compression Taxonomies.

❖ **Student Tribute (Page 86):** Premium "Thank You, Future Data Scientists!" dedication certificate.

# Multiple Choice Master Key

Quick reference key for Questions 1 through 90 with priority star ratings. Complete worked solutions begin on the next page.

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| Q1 **A** ★★★★★ | Q2 **C** ★★★★★ | Q3 **D** ★★★★★ | Q4 **D** ★★★★★ | Q5 **A** ★★★★ | Q6 **C** ★★★ | Q7 **C** ★★★★ | Q8 **B** ★★★★★ | Q9 **D** ★★★ | Q10 **D** ★★★★ |
| Q11 **C** ★★★★★ | Q12 **C** ★★★★★ | Q13 **A** ★★★★ | Q14 **A** ★★★★★ | Q15 **D** ★★★★ | Q16 **D** ★★★★ | Q17 **C** ★★★★★ | Q18 **A** ★★★ | Q19 **A** ★★★ | Q20 **B** ★★★★ |
| Q21 **C** ★★★★★ | Q22 **D** ★★★★★ | Q23 **A** ★★★★★ | Q24 **A** ★★★★ | Q25 **B** ★★★★ | Q26 **C** ★★★★★ | Q27 **C** ★★★★★ | Q28 **D** ★★★★ | Q29 **A** ★★★ | Q30 **B** ★★★★ |
| Q31 **C** ★★★★★ | Q32 **C** ★★★★★ | Q33 **A** ★★★★★ | Q34 **D** ★★★★ | Q35 **B** ★★★★★ | Q36 **C** ★★★★ | Q37 **A** ★★★★ | Q38 **A** ★★★ | Q39 **B** ★★★ | Q40 **D** ★★★★★ |
| Q41 **C** ★★★★★ | Q42 **A** ★★★★★ | Q43 **A** ★★★★★ | Q44 **B** ★★★★★ | Q45 **B** ★★★★ | Q46 **D** ★★★★ | Q47 **A** ★★★★ | Q48 **B** ★★★★★ | Q49 **B** ★★★ | Q50 **C** ★★★★ |
| Q51 **C** ★★★★★ | Q52 **D** ★★★★★ | Q53 **D** ★★★★★ | Q54 **B** ★★★★ | Q55 **C** ★★★★★ | Q56 **C** ★★★★ | Q57 **A** ★★★★ | Q58 **D** ★★★ | Q59 **D** ★★★★ | Q60 **C** ★★★★ |
| Q61 **C** ★★★★★ | Q62 **B** ★★★★★ | Q63 **B** ★★★★★ | Q64 **D** ★★★★ | Q65 **D** ★★★★★ | Q66 **C** ★★★★ | Q67 **B** ★★★★ | Q68 **B** ★★★ | Q69 **A** ★★★★★ | Q70 **D** ★★★★ |
| Q71 **D** ★★★★★ | Q72 **B** ★★★★★ | Q73 **B** ★★★★ | Q74 **A** ★★★★★ | Q75 **C** ★★★★★ | Q76 **D** ★★★★ | Q77 **D** ★★★★ | Q78 **A** ★★★ | Q79 **A** ★★★★★ | Q80 **C** ★★★ |
| Q81 **B** ★★★★★ | Q82 **B** ★★★★★ | Q83 **D** ★★★★★ | Q84 **A** ★★★★★ | Q85 **C** ★★★★ | Q86 **B** ★★★★ | Q87 **B** ★★★★ | Q88 **A** ★★★ | Q89 **D** ★★★★ | Q90 **C** ★★★★★ |

| 90 | 100% | 3-Pass | 10 FRQs |
|---|---|---|---|
| TOTAL MCQS | CED ALIGNED | VERIFIED | 90 PTS TOTAL |

### Question 1: Binary to Decimal Conversion

★★★★★ VERY IMPORTANT

**Correct Answer: (A) — 166**

**STEP-BY-STEP EXPLANATION**

- To convert an 8-bit binary number to decimal, evaluate each bit's positional place value (powers of 2 from right to left: 128, 64, 32, 16, 8, 4, 2, 1).
- Assign place values to the bits of $10100110_2$:
- • 1 × 128 = 128
- • 0 × 64 = 0
- • 1 × 32 = 32
- • 0 × 16 = 0
- • 0 × 8 = 0
- • 1 × 4 = 4
- • 1 × 2 = 2
- • 0 × 1 = 0
- Sum the active components: 128 + 32 + 4 + 2 = 166.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(B):** Omits the 4 place value (128 + 32 + 2 = 162).

✗ **(C):** Incorrectly computes 128 + 16 + 4 + 2 (confusing the 32 position with 16), yielding 150.

✗ **(D):** Incorrectly adds the 8 place value (128 + 32 + 8 + 4 + 2 = 174), but bit position 4 (value 8) is 0.

### Question 2: Decimal to Binary Conversion

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — 01011001**

**STEP-BY-STEP EXPLANATION**

- Convert 89 to binary by finding the largest powers of 2 that sum to 89:
- • 89 >= 64 -> subtract 64, remainder 25 (Bit 64 is 1).
- • 25 < 32 -> (Bit 32 is 0).
- • 25 >= 16 -> subtract 16, remainder 9 (Bit 16 is 1).
- • 9 >= 8 -> subtract 8, remainder 1 (Bit 8 is 1).
- • 1 < 4 -> (Bit 4 is 0).
- • 1 < 2 -> (Bit 2 is 0).
- • 1 >= 1 -> subtract 1, remainder 0 (Bit 1 is 1).
- Combine the bits with place values [128, 64, 32, 16, 8, 4, 2, 1]: $01011001_2$.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** 01001101 represents 64 + 8 + 4 + 1 = 77.

✗ **(B):** 01101001 represents 64 + 32 + 8 + 1 = 105.

✗ **(D):** 01011011 represents 64 + 16 + 8 + 2 + 1 = 91, which is 2 greater than 89.

### Question 3: Number of Values Represented by N Bits

★★★★★ VERY IMPORTANT

**Correct Answer: (D) — 448 more user IDs**

**STEP-BY-STEP EXPLANATION**

- The formula for the total number of unique values that can be represented by n bits is $2^n$.
- • With 6 bits: $2^6$ = 64 distinct values (from 0 to 63).
- • With 9 bits: $2^9$ = 512 distinct values (from 0 to 511).
- • To find how many *more* user IDs can be represented: 512 − 64 = 448.
- Each additional bit doubles the total number of representable values.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** 512 represents the new total capacity with 9 bits, not the increase (how many *more* IDs).

✗ **(B):** Confuses the number of additional bits (9 − 6 = 3 bits) with the number of representable data values.

✗ **(C):** 64 represents the original capacity with 6 bits.

### Question 4: Minimum Bits Required for Target Range

★★★★★ VERY IMPORTANT

**Correct Answer: (D) — 11 bits**

STEP-BY-STEP EXPLANATION

- To find the minimum number of bits n needed to represent N distinct items, find the smallest integer n such that $2^n >= N$.
- Evaluate powers of 2 around 1,250:
- • $2^{10}$ = 1,024 (Insufficient: only covers up to 1,024 students, leaving 226 without unique IDs).
- • $2^{11}$ = 2,048 (Sufficient: 2,048 >= 1,250).
- Therefore, a minimum of 11 bits is required.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** 8 bits only provides $2^8$ = 256 unique IDs.
✗ **(B):** 10 bits can only represent $2^{10}$ = 1,024 unique values, which is fewer than 1,250.
✗ **(C):** 12 bits provides 4,096 values, which can represent 1,250 students but is not the *minimum* required.

### Question 5: Bit Place Values and Doubling Principle

★★★★ HIGH PRIORITY

**Correct Answer: (A) — The total number of representable values doubles.**

STEP-BY-STEP EXPLANATION

- In a base-2 positional number system, each bit can hold one of two states: 0 or 1.
- For n bits, there are $2^n$ possible combinations.
- When adding 1 bit: $2^{n+1} = 2^n \times 2$.
- Thus, every single bit added to a binary number exactly doubles the total quantity of distinct patterns or values that can be encoded.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** The total number does not square (e.g., $2^3$ = 8; $2^4$ = 16, which is not $8^2$ = 64).
✗ **(C):** Adding 1 bit multiplies the total count by 2; it does not simply add 2 (e.g., going from 3 bits (8) to 4 bits (16) adds 8, not 2).
✗ **(D):** The maximum value approximately doubles, rather than increasing by only 1.

### Question 6: Binary Number Comparison

★★★ MANDATORY

**Correct Answer: (C) — P < R < Q**

STEP-BY-STEP EXPLANATION

- Convert each binary number to decimal or compare bit-by-bit from left to right:
- • P = $01101100_2$ = 64 + 32 + 8 + 4 = 108.
- • Q = $01110001_2$ = 64 + 32 + 16 + 1 = 113.
- • R = $01101111_2$ = 64 + 32 + 8 + 4 + 2 + 1 = 111.
- Comparing the decimal values: 108 < 111 < 113, which corresponds to P < R < Q.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Incorrectly assumes Q is smaller than R, but Q has a 1 in the 16s place while R has a 0 in the 16s place.
✗ **(B):** Inverts the order by placing Q (the largest at 113) as the smallest.
✗ **(D):** Places R as the smallest, but P (108) is smaller than R (111).

### Question 7: Odd and Even Binary Numbers

★★★★ HIGH PRIORITY

**Correct Answer: (C) — Examine the rightmost bit (least significant bit): if it is 1, the number is odd; if it is 0, the number is even.**

STEP-BY-STEP EXPLANATION

- In binary representation, place values are powers of 2: [..., 64, 32, 16, 8, 4, 2, 1].
- Every place value greater than $2^0$ is an even number (2, 4, 8, 16, 32, ...).
- The sum of any combination of even numbers is always even.
- Therefore, the only bit that can contribute an odd quantity (1) to the total sum is the least significant bit ($2^0 = 1$).
- If the rightmost bit is 1, the total value is odd; if it is 0, the total value is even.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** The leftmost bit indicates the largest power of 2, which is an even number (for any sequence with more than 1 bit) and does not govern odd/even parity.

✗ **(B):** The bit length of the representation has no relation to the mathematical parity of the encoded integer.

✗ **(D):** The total number of 1s determines Hamming weight/parity bits for error checking, not whether the numeric value itself is mathematically odd or even (e.g., $6 = 110_2$ has two 1s, yet is even; $3 = 11_2$ has two 1s, yet is odd).

### Question 8: Maximum Unsigned Value with N Bits

★★★★★ VERY IMPORTANT

**Correct Answer: (B) — 255**

STEP-BY-STEP EXPLANATION

- An unsigned n-bit binary number can represent $2^n$ total unique values, ranging from 0 to $2^n - 1$.
- For n = 8 bits:
- • Total combinations: $2^8 = 256$.
- • The values start at 0 ($00000000_2$).
- • The maximum value occurs when all 8 bits are 1 ($11111111_2$): 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255.
- Thus, 255 is the maximum representable decimal value.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** 128 is $2^7$, the value of the highest single bit position in an 8-bit byte.

✗ **(C):** 127 is the maximum positive value in an 8-bit *signed* two's complement integer, not unsigned.

✗ **(D):** 256 is the *total count* of distinct values (including 0), not the maximum value. The range is 0 to 255.

### Question 9: Binary Addition Mechanics

★★★ MANDATORY

**Correct Answer: (D) — $1011_2$**

STEP-BY-STEP EXPLANATION

- Perform column-by-column binary addition from right to left:
- 0 1 1 0 (decimal 6)
- + 0 1 0 1 (decimal 5)
- ---------
- • Column 1 (units): 0 + 1 = 1 (no carry).
- • Column 2 (twos): 1 + 0 = 1 (no carry).
- • Column 3 (fours): 1 + 1 = 0 (carry 1 to eights).
- • Column 4 (eights): 0 + 0 + 1 (carry) = 1.
- Result: $1011_2$.
- Verification in decimal: 6 + 5 = 11. In binary, $1011_2 = 8 + 2 + 1 = 11$.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** $1111_2$ represents decimal 15.

✗ **(B):** $1100_2$ represents decimal 12, an arithmetic error.

✗ **(C):** $1001_2$ represents decimal 9, failing to include the twos place.

### Question 10: Bit Representation of Discrete States

★★★★ HIGH PRIORITY

**Correct Answer: (D) — 3 bits**

**STEP-BY-STEP EXPLANATION**

- To represent k distinct states, we need n bits such that $2^n >= k$.
- • If n = 2 bits: $2^2 = 4$ states (insufficient for 5 states).
- • If n = 3 bits: $2^3 = 8$ states (sufficient because 8 >= 5).
- Therefore, a minimum of 3 bits is required.
- Three bits can represent up to 8 states (e.g., 000, 001, 010, 011, 100), leaving 3 unused combinations for future expansion.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** 4 bits represents up to 16 states, which is also more than the minimum of 3 bits.

✗ **(B):** 5 bits could represent $2^5 = 32$ states, which is valid but not the *minimum* required.

✗ **(C):** 2 bits can only represent $2^2 = 4$ distinct states, leaving one state unencoded.

### Question 11: Hexadecimal to Binary Conversion

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — 01001101**

**STEP-BY-STEP EXPLANATION**

- In hexadecimal (base-16), each digit directly corresponds to a 4-bit binary sequence (nibble):
- • $4_{16}$ in binary = $0100_2$ (0×8 + 1×4 + 0×2 + 0×1 = 4).
- • In hex, A=10, B=11, C=12, D=13, E=14, F=15.
- • Therefore, $D_{16}$ represents decimal 13.
- • 13 in binary = $1101_2$ (1×8 + 1×4 + 0×2 + 1×1 = 13).
- Combine the two nibbles: 0100 $1101_2$ = $01001101_2$.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** 01001110 represents $4E_{16}$ (4 in first nibble, 14 in second nibble).

✗ **(B):** 00101101 represents $2D_{16}$ (2 in first nibble instead of 4).

✗ **(D):** 01001011 represents $4B_{16}$ (B=11 in second nibble instead of D=13).

### Question 12: Binary to Hexadecimal Conversion

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — $BE_{16}$**

**STEP-BY-STEP EXPLANATION**

- To convert an 8-bit binary number to hexadecimal, divide the bits into two 4-bit groups (nibbles):
- • First nibble: $1011_2$
- - Calculation: 8 + 0 + 2 + 1 = 11.
- - Decimal 11 in hexadecimal is 'B'.
- • Second nibble: $1110_2$
- - Calculation: 8 + 4 + 2 + 0 = 14.
- - Decimal 14 in hexadecimal is 'E'.
- Combine the two hex digits: $BE_{16}$.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** $BD_{16}$ represents $10111101_2$ (second nibble is 13, not 14).

✗ **(B):** $BF_{16}$ represents $10111111_2$ (second nibble is 15 = F, not 14 = E).

✗ **(D):** $AE_{16}$ represents $10101110_2$ (first nibble is 10 = A, not 11 = B).

### Question 13: Purpose of Hexadecimal in Computing

★★★★ HIGH PRIORITY

**Correct Answer: (A) — Hexadecimal is much more concise and human-readable, reducing long sequences of bits while maintaining an exact, direct 1-to-4 mapping with binary.**

STEP-BY-STEP EXPLANATION

- Because 16 is a power of 2 ($2^4 = 16$), exactly four binary bits map cleanly to a single hexadecimal digit.
- An 8-bit byte (e.g., $11111111_2$) requires only two hex digits (FF), and a 32-bit memory address requires only 8 hex digits instead of 32 ones and zeros.
- This compactness makes hex easier for humans to read, transcribe, and debug, with zero ambiguity during conversion.
- Internally, all computer hardware still operates exclusively on binary voltages.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Physical hardware transistors and logic gates operate strictly on two voltage states (binary 0 and 1), never directly on base-16.
✗ **(C):** Hexadecimal is a standard, open numeral representation, not an encryption or security mechanism.
✗ **(D):** Hexadecimal is merely an alternate notation; it does not increase the physical storage density of the silicon.

### Question 14: RGB Hexadecimal Color Decoding

★★★★★ VERY IMPORTANT

**Correct Answer: (A) — Pure bright green**

STEP-BY-STEP EXPLANATION

- Standard 24-bit RGB color representation uses three pairs of hexadecimal digits formatted as `#RRGGBB`:
- • `RR` represents the red channel intensity ($00_{16} = 0$, $FF_{16} = 255$).
- • `GG` represents the green channel intensity ($00_{16} = 0$, $FF_{16} = 255$).
- • `BB` represents the blue channel intensity ($00_{16} = 0$, $FF_{16} = 255$).
- In `#00FF00`, Red = 0, Green = 255 (maximum), and Blue = 0.
- This illuminates only the green sub-pixels at full intensity, producing pure bright green.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Dark gray requires all three channels to be balanced at a low non-zero level, such as `#333333`.
✗ **(C):** Pure blue is represented by `#0000FF`.
✗ **(D):** Pure red is represented by `#FF0000`.

### Question 15: RGB Additive Color Mixing in Hex

★★★★ HIGH PRIORITY

**Correct Answer: (D) — `#FFFF00`, producing bright yellow**

STEP-BY-STEP EXPLANATION

- In the additive RGB color model used by digital displays (monitors, phones):
- • Red channel at max = `FF`.
- • Green channel at max = `FF`.
- • Blue channel at zero = `00`.
- Combining Red + Green produces `#FFFF00`.
- In additive light optics, mixing red and green wavelengths produces yellow light.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** `#FFFFFF` is Red + Green + Blue all at maximum, producing white.
✗ **(B):** `#FF00FF` is Red + Blue, which produces magenta.
✗ **(C):** `#00FFFF` is Green + Blue, which produces cyan.

### Question 16: ASCII Character Encoding Standards

★★★★ HIGH PRIORITY

**Correct Answer: (D) — $1000011_2$**

**STEP-BY-STEP EXPLANATION**

- In ASCII, consecutive alphabetical characters are assigned consecutive numeric codes:
- • 'A' = decimal 65 = $1000001_2$.
- • 'B' = decimal 66 = $1000010_2$.
- • 'C' = decimal 67.
- Convert decimal 67 to 7-bit binary:
- • 67 = 64 + 2 + 1.
- • Place values [64, 32, 16, 8, 4, 2, 1] -> 1 0 0 0 0 1 $1_2$.
- Therefore, 'C' is encoded as $1000011_2$.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** $1000010_2$ is decimal 66, which represents the letter 'B'.

✗ **(B):** $1000100_2$ is decimal 68, which represents the letter 'D'.

✗ **(C):** $0100011_2$ is decimal 35, which represents the '#' symbol.

### Question 17: ASCII vs. Unicode Comparison

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — Unicode uses a variable/larger bit-width capable of representing over a million unique characters, enabling support for all global languages, mathematical symbols, and emojis.**

**STEP-BY-STEP EXPLANATION**

- Standard ASCII uses 7 bits (128 characters) or extended ASCII uses 8 bits (256 characters), which only accommodates English letters, numbers, and basic punctuation.
- Unicode was created to unify digital text worldwide by assigning unique code points across up to 32 bits, accommodating over 1.1 million distinct characters.
- This enables computers to seamlessly encode text in Chinese, Arabic, Hindi, Cyrillic, Greek, mathematical notation, and modern emojis.
- UTF-8 is backward-compatible with 7-bit ASCII while expanding dynamically for other global scripts.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** Unicode does not reduce the bit-width of standard English characters; in UTF-8, standard English characters still consume 8 bits (1 byte), identical to extended ASCII.

✗ **(B):** Unicode remains strictly digital (binary); it does not use analog signals.

✗ **(D):** A 7-bit system can only ever hold 128 items ($2^7 = 128$), which is physically incapable of encoding thousands of international characters.

### Question 18: String Character vs. Numeric Value Representation

★★★ MANDATORY

**Correct Answer: (A) — The integer 5 is stored as its binary numeric equivalent ($00000101_2$), while the character '5' is stored as its ASCII/Unicode character code ($00110101_2$).**

**STEP-BY-STEP EXPLANATION**

- Data type dictates the binary interpretation of bits in memory:
- • As an integer, 5 is stored using positional binary: $5_{10} = 00000101_2$ (in an 8-bit byte).
- • As a text character, '5' is encoded according to the ASCII table, where the numeral '5' has code point $53_{10}$.
- • Decimal 53 in binary is 32 + 16 + 4 + 1 = $00110101_2$.
- Because their binary representations differ completely, arithmetic operations like addition behave differently than string concatenation.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(B):** They do not share the same bit pattern; '5' uses ASCII code 53 ($00110101_2$), whereas integer 5 is numeric 5 ($00000101_2$).

✗ **(C):** Character length in memory is determined by the encoding standard (e.g., 8 bits for UTF-8/ASCII), not the numerical value of the digit.

✗ **(D):** Characters are stored as numeric binary code points, not as vector graphics in RAM.

### Question 19: Hexadecimal Counting Sequence

★★★ MANDATORY

**Correct Answer: (A) — $30_{16}$**

**STEP-BY-STEP EXPLANATION**

- Follow hexadecimal counting rules (0, 1, ..., 9, A, B, C, D, E, F):
- • Initial value: $2E_{16}$.
- • Increment 1: $2E_{16}$ + 1 = $2F_{16}$ (since F is the digit after E).
- • Increment 2: $2F_{16}$ + 1.
- - Because F is the maximum single digit in base-16, adding 1 causes the units digit to roll over to 0 and carries 1 into the 16s place.
- - 2 + 1 = 3.
- • Result: $30_{16}$.
- Verification in decimal: $2E_{16}$ = 2×16 + 14 = 46. Adding 2 gives 48. 48 / 16 = 3 remainder 0 -> $30_{16}$.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(B):** The letter 'G' does not exist in hexadecimal; base-16 digits terminate at 'F'.

✗ **(C):** $2F_{16}$ is the result after only 1 increment, not 2.

✗ **(D):** $31_{16}$ represents 49 in decimal, which is an increment of 3.

### Question 20: Hexadecimal String Length and Byte Storage

★★★★ HIGH PRIORITY

**Correct Answer: (B) — 4 bytes**

**STEP-BY-STEP EXPLANATION**

- Every hexadecimal digit corresponds to exactly 4 bits (1 nibble).
- Since 1 byte = 8 bits, it takes exactly 2 hexadecimal digits to represent 1 byte (2 × 4 bits = 8 bits).
- Count the hexadecimal digits in `A3F09C8B`:
- • Total digits = 8 hex characters.
- • Total bits = 8 digits × 4 bits/digit = 32 bits.
- • Total bytes = 32 bits / 8 bits/byte = 4 bytes.
- Therefore, the string represents exactly 4 bytes.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** 2 bytes would only correspond to 4 hex digits (16 bits).

✗ **(C):** 16 bytes would require 32 hex characters.

✗ **(D):** 8 is the number of hexadecimal *characters*, but each byte requires 2 hex characters.

### Question 21: Definition and Root Cause of Integer Overflow

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — A calculation produces a numerical value that exceeds the maximum value that can be represented within the allocated fixed number of bits.**

**STEP-BY-STEP EXPLANATION**

- In digital computing, memory registers allocate a predetermined, fixed number of bits (such as 8, 16, 32, or 64 bits) to store integer variables.
- A fixed number of bits n can only represent a finite range of numbers (e.g., 0 to $2^n - 1$ for unsigned numbers).
- When an arithmetic operation (such as addition or multiplication) yields a value strictly larger than this maximum limit, the additional carry bit cannot fit into the allocated register.
- This condition is defined by the College Board (DAT-1.B) as an integer overflow error.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** Clock frequency jitter is a physical hardware defect, whereas overflow is a fundamental mathematical consequence of finite bit width.

✗ **(B):** Referencing uninitialized variables results in a runtime error or undefined variable lookup, not an overflow.

✗ **(D):** Division by zero produces a ZeroDivisionError/runtime trap, not an integer overflow.

### Question 22: 4-Bit Register Overflow Mechanics

★★★★★ VERY IMPORTANT

**Correct Answer: (D) — An overflow occurs, and the register resets to $0000_2$ because the fifth bit cannot be stored in the 4-bit register.**

**STEP-BY-STEP EXPLANATION**

- An unsigned 4-bit binary number can only represent values from 0 ($0000_2$) to 15 ($1111_2$).
- Adding 1 to $1111_2$ yields $10000_2$ in 5-bit binary arithmetic.
- Because the register only has 4 storage bits, the leading carry bit (the 16s place) is discarded or moved to an overflow flag.
- The 4 bits remaining in the register are $0000_2$ (decimal 0).
- This wraparound effect is the classic symptom of an unhandled integer overflow.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** Hardware registers have fixed physical transistor widths; they cannot automatically manufacture additional physical bit circuits at runtime.
✗ **(B):** Standard binary addition rolls over to $0000_2$ unless custom clamping logic is explicitly programmed.
✗ **(C):** The register simply wraps around to $0000_2$; standard integer addition does not physically destroy or permanently brick the hardware.

### Question 23: 8-Bit Unsigned Integer Overflow

★★★★★ VERY IMPORTANT

**Correct Answer: (A) — The game stores the score in an unsigned 8-bit integer, where 255 ($11111111_2$) is the maximum representable value; adding 1 causes the 9th bit to be discarded, leaving $00000000_2$.**

**STEP-BY-STEP EXPLANATION**

- An unsigned 8-bit byte has a maximum capacity of $2^8 - 1 = 255$ (binary $11111111_2$).
- When adding 1 to 255:
- $11111111_2 + 00000001_2 = 1\ 00000000_2$.
- Because only 8 bits of memory are allocated, the 9th bit (256) is truncated, leaving $00000000_2$ (decimal 0).
- This is an integer overflow error resulting in a score rollover.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(B):** Score counters are stored as integers, not floating-point numbers; round-off errors apply to fractional real numbers, not integer rollover.
✗ **(C):** Screen refresh rate governs visual display timing, not the internal binary value of memory variables.
✗ **(D):** Controller inputs send button state events, not destructive memory wipe codes.

### Question 24: Signed Two's Complement Range and Overflow

★★★★ HIGH PRIORITY

**Correct Answer: (A) — -128**

**STEP-BY-STEP EXPLANATION**

- In 8-bit signed two's complement representation:
- • Positive numbers have a leading sign bit of 0 (range 0 to +127).
- • Negative numbers have a leading sign bit of 1 (range -128 to -1).
- The binary representation of +127 is $01111111_2$.
- Adding 1:
- $01111111_2 + 1 = 10000000_2$.
- In two's complement, $10000000_2$ represents the most negative number: -128.
- This causes an overflow where incrementing the maximum positive number abruptly wraps around into the minimum negative number.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(B):** 0 is represented as $00000000_2$, which is not the result of $01111111_2 + 1$.
✗ **(C):** +128 cannot be represented in an 8-bit signed integer because the positive range stops at +127.
✗ **(D):** -1 is represented as $11111111_2$ in two's complement.

### Question 25: Historical Video Game Glitch and 8-Bit Overflow

★★★★ HIGH PRIORITY

**Correct Answer: (B) — An 8-bit level counter variable overflowed when incrementing past 255, causing the subroutine drawing bonus fruit to enter an infinite loop drawing 256 fruit items instead of 7.**

**STEP-BY-STEP EXPLANATION**

- The fruit-drawing subroutine in Pac-Man calculated how many fruits to display using an 8-bit register holding the current level number.
- When the player reached level 256, the 8-bit level counter overflowed from 255 back to 0 (256 mod 256 = 0).
- A loop designed to draw fruit until the counter reached 0 decremented the level number, underflowed, and attempted to draw 256 fruit symbols across the screen.
- This overwrote video RAM with random character sprite data, creating the classic split-screen glitch.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** The kill screen was an unintended software bug, not an intentional cryptographic security feature.
✗ **(C):** The glitch is purely software-based in video memory, not a physical hardware failure of the cathode-ray tube.
✗ **(D):** 1980 arcade cabinets operated entirely offline on localized ROM boards; no internet connection existed.

### Question 26: Ariane 5 Rocket Failure & Type Conversion Overflow

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — An integer overflow error during numerical type conversion from a wide data type to a narrow data type.**

**STEP-BY-STEP EXPLANATION**

- A 16-bit signed integer can only store numbers between -32,768 and +32,767 ($2^{15} - 1$).
- The rocket's horizontal velocity measurement was a 64-bit floating-point value that exceeded 32,767.
- When the software attempted to convert this large float into a 16-bit signed integer without error checking, an integer overflow occurred.
- The processor triggered a diagnostic hardware exception, which was interpreted by the flight computers as a failure, leading the rocket to self-destruct.
- This is one of the most famous real-world examples of integer overflow in engineering history.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** The software had compiled and run successfully in the predecessor Ariane 4 rocket; it was a runtime overflow defect, not a syntax error.
✗ **(B):** Unit conversion errors occurred in the Mars Climate Orbiter (metric vs English units in 1999), not the Ariane 5 rocket.
✗ **(D):** Lossy compression refers to multimedia file size reduction (like JPEG or MP3), which had nothing to do with internal rocket trajectory variables.

### Question 27: The Year 2038 Problem (Unix Epoch Overflow)

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — The signed 32-bit counter will overflow into negative numbers (-2,147,483,648), causing systems to interpret the date as December 13, 1901.**

**STEP-BY-STEP EXPLANATION**

- A 32-bit signed integer has a maximum value of $2^{31} - 1 = 2,147,483,647$.
- At 03:14:07 UTC on January 19, 2038, the seconds counter reaches this exact limit.
- One second later, an integer overflow occurs: the sign bit flips from 0 to 1, changing the stored value to -2,147,483,648.
- Software interpreting negative epoch times will calculate the current date as December 13, 1901, causing critical failures in scheduling, financial transactions, and cryptographic certificates.
- Modern operating systems are migrating to 64-bit time integers (which will not overflow for hundreds of billions of years).

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** AM/PM format is a display layer convention; the underlying timestamp representation is an epoch second counter.
✗ **(B):** Clock hardware frequency does not accelerate to compensate for integer overflow.
✗ **(D):** Without upgrading to 64 bits, the fixed 32-bit register cannot represent values beyond 2,147,483,647; it will wrap around to 1901.

### Question 28: Preventing Integer Overflow Defects

★★★★ HIGH PRIORITY

**Correct Answer: (D) — Use 64-bit integers or arbitrary-precision integer libraries ('BigInt') that dynamically allocate memory as numbers grow.**

STEP-BY-STEP EXPLANATION

- A 32-bit integer overflows at approximately 2.14 billion ($21.4 million if stored in cents).
- Switching to 64-bit integers expands the maximum value to over 9 quintillion ($9 \times 10^{18}$), virtually eliminating overflow for financial amounts.
- Additionally, utilizing arbitrary-precision libraries (such as `BigInt` or `BigDecimal`) dynamically allocates additional bytes as numbers grow, preventing overflow completely.
- College Board Big Idea 2 emphasizes allocating sufficient bit-widths to match domain requirements.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Converting to ASCII strings does not resolve arithmetic register limitations and introduces significant computational overhead.
✗ **(B):** Lossy compression discards data permanently, which would destroy financial precision and result in illegal accounting discrepancies.
✗ **(C):** Rounding financial amounts introduces catastrophic accounting errors and fails legal compliance.

### Question 29: Mechanical Odometer Analogy for Overflow

★★★ MANDATORY

**Correct Answer: (A) — It represents an overflow error in a fixed-width base-10 system, perfectly illustrating how fixed-width registers wrap around when their physical capacity is exceeded.**

STEP-BY-STEP EXPLANATION

- The 5-dial odometer has a fixed capacity of $10^5$ = 100,000 states (00,000 to 99,999).
- When 1 mile is added to 99,999, the sum is 100,000. Because there is no 6th dial to display the 1, it wraps around to 00,000.
- This is the exact decimal equivalent of an unsigned binary register overflowing (e.g., an 8-bit register rolling from 255 to 0).
- In both systems, the leading carry digit is lost due to fixed physical capacity.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Round-off errors occur with fractional numbers; the odometer is counting discrete whole miles.
✗ **(C):** No compression algorithm is operating; it is a fixed-width register limit.
✗ **(D):** The mechanism is operating exactly as designed; it is a structural capacity limitation, not a mechanical syntax flaw.

### Question 30: Overflow vs. Round-off vs. Logic Errors

★★★★ HIGH PRIORITY

**Correct Answer: (B) — An uncompressed video file counter adds 1 to an integer variable currently storing $2^{32} - 1$, resulting in an unexpected value of 0.**

STEP-BY-STEP EXPLANATION

- Option A describes an **overflow error**: exceeding the maximum integer representable by a 32-bit register ($2^{32} - 1$) causes it to roll over to 0.
- Option B describes a **round-off error**: imprecision in representing non-terminating fractional real numbers in binary floating point.
- Option C describes a **logic error**: the code runs without crashing but uses the wrong mathematical operator (+ instead of *).
- Option D describes a **logic error**: an incorrect boolean condition producing an infinite loop.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** This is an algorithmic logic error.
✗ **(C):** This is a floating-point round-off error, not an integer overflow.
✗ **(D):** This is a logic error in control flow.

### Question 31: Definition and Cause of Round-Off Errors

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — Real fractional numbers have infinite precision or non-terminating binary expansions, but computer memory allocates only a finite number of bits to store them.**

STEP-BY-STEP EXPLANATION

- In mathematics, real numbers exist on a continuous continuum with infinite precision (infinite decimal/fractional digits).
- Digital computers, however, have finite physical memory and represent floating-point numbers using a fixed number of bits (such as 32 bits for single precision or 64 bits for double precision under IEEE 754).
- Because many common fractions (like 1/3 or 1/10) cannot be represented as terminating sums of negative powers of 2 (1/2, 1/4, 1/8, 1/16, etc.), they must be approximated and rounded off.
- This inherent mathematical limitation creates floating-point round-off error.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Grammar or spelling mistakes are syntax errors, which are caught at compile time and unrelated to fractional binary representation.
✗ **(B):** Lossy compression refers to multimedia file encoding (such as JPEG/MP3), not mathematical variable storage in RAM.
✗ **(D):** Integer variables exceeding register limits describes an **overflow error**, not a round-off error.

### Question 32: Why 0.1 Repeats in Binary Representation

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — A base-10 terminating fraction can only terminate in binary if its denominator's prime factors are exclusively powers of 2; because 10 has a prime factor of 5, 1/10 cannot terminate in base-2.**

STEP-BY-STEP EXPLANATION

- In any positional number system of base b, a reduced fraction p/q terminates if and only if all prime factors of the denominator q divide base b.
- In base-10, 10 = 2 × 5, so fractions whose denominators contain only factors of 2 and 5 (like 1/2, 1/4, 1/5, 1/10) terminate.
- In base-2, the only prime factor is 2. Therefore, only fractions whose denominators are pure powers of 2 (1/2, 1/4, 1/8, 1/16, etc.) terminate in binary.
- Because the denominator 10 contains the factor 5, 1/10 produces an infinite repeating binary sequence ($0.0001100110011\ldots_2$), requiring truncation in finite memory.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** ASCII encodes text characters (letters, digits, symbols), not numerical floating-point arithmetic.
✗ **(B):** Byte width does not dictate the mathematical prime factorization of positional number systems.
✗ **(D):** Computers divide floating-point numbers below 1 routinely using IEEE 754 floating-point ALUs.

### Question 33: Floating-Point Equality Comparison Pitfall

★★★★★ VERY IMPORTANT

**Correct Answer: (A) — Due to floating-point round-off error, 0.1 + 0.2 evaluates to approximately 0.30000000000000004 in binary floating point, which is not strictly equal to the binary approximation of 0.3.**

STEP-BY-STEP EXPLANATION

- Because 0.1, 0.2, and 0.3 are all non-terminating repeating fractions in binary, they are stored as finite 64-bit approximations in memory.
- When adding the binary representations of 0.1 and 0.2, the combined rounding discrepancies yield 0.30000000000000004440892098500626...
- When the strict equality operator `==` compares this stored sum to the stored representation of 0.3, the binary bits do not match identically.
- Consequently, the expression evaluates to `false` and the program branches to the ELSE block displaying 'NOT EQUAL'.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Conditional statements evaluate floating-point expressions routinely; they do not crash on decimal points.
✗ **(C):** These are floating-point fractional numbers, not integer overflows.
✗ **(D):** Standard numeric addition is performed; string concatenation only occurs when inputs are string literals (`"0.1" + "0.2"`).

### Question 34: Accumulation of Round-Off Error in Repetitive Loops

★★★★ HIGH PRIORITY

**Correct Answer: (D) — Small floating-point round-off errors in each individual addition step accumulated over one million iterations, compounding into a measurable deviation.**

**STEP-BY-STEP EXPLANATION**

- In numerical analysis, each individual floating-point operation introduces a microscopic rounding error (machine epsilon, on the order of $10^{-16}$ for 64-bit doubles).
- While a single round-off error of $10^{-16}$ is negligible, performing millions or billions of repeated additions causes these tiny discrepancies to accumulate linearly or exponentially.
- Over 1,000,000 iterations, the tiny fractional error compounded, resulting in an end total of 999.999824 instead of exactly 1000.0.
- This demonstrates why numerical algorithms must be designed to minimize cumulative round-off drift.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** Thermal throttling reduces CPU clock speed to prevent overheating; it never causes mathematical errors in calculations.
✗ **(B):** Compilers do not apply lossy multimedia compression to active variables in RAM.
✗ **(C):** 1,000,000 fits comfortably inside a standard 32-bit integer (max ~2.14 billion); no integer overflow occurred.

### Question 35: 1991 Patriot Missile System Round-Off Catastrophe

★★★★★ VERY IMPORTANT

**Correct Answer: (B) — The round-off error accumulated to approximately 0.34 seconds; at supersonic speeds, this caused the tracking system to miscalculate the missile's position by over 500 meters.**

**STEP-BY-STEP EXPLANATION**

- The Patriot missile's tracking software calculated time by multiplying an integer clock count by 1/10 (0.1).
- Because 1/10 is a repeating fraction in binary, truncating it to 24 bits introduced an error of 0.000000095 seconds per 1/10th of a second.
- After 100 hours of continuous operation (100 × 3600 × 10 = 3,600,000 ticks):
- 3,600,000 × 0.000000095 ≈ 0.3433 seconds.
- An incoming Scud missile traveled at approximately 1,676 meters per second.
- In 0.3433 seconds, the missile traveled over 575 meters, placing it completely outside the radar tracking gate, so the system never attempted an intercept.
- This is one of the most tragic real-world illustrations of cumulative floating-point round-off error.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** The radar operated continuously for 100 hours; it was a numerical timing drift, not a syntax error.
✗ **(C):** The failure was caused by tracking radar clock drift in software, not a physical rocket fuel depletion or fuel gauge overflow.
✗ **(D):** The battery system was continuously powered by military generators; it did not suffer power loss.

### Question 36: Safe Floating-Point Comparison with Epsilon

★★★★ HIGH PRIORITY

**Correct Answer: (C) — Check whether the absolute difference between `a` and `b` is less than a small predefined threshold (epsilon): `abs(a - b) < epsilon`.**

**STEP-BY-STEP EXPLANATION**

- Due to floating-point round-off errors, two numbers that are theoretically equal may differ by a tiny fraction in their least significant bits (e.g., 0.30000000000000004 vs. 0.30000000000000000).
- Using `a == b` fails whenever this microscopic discrepancy exists.
- The robust solution is tolerance-based comparison: determine if `|a − b| < ε` (epsilon), where epsilon is a small threshold (e.g., 0.00001 or 1e-9).
- If the difference is smaller than epsilon, the values are considered practically equal.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** Strict equality `a == b` fails frequently with floating-point calculations due to binary rounding imprecision.
✗ **(B):** Converting to 8-bit unsigned integers discards all fractional data and caps the number at 255, destroying precision and range.
✗ **(D):** Rounding to whole integers destroys all decimal precision (e.g., 4.1 and 4.4 would be deemed equal, while 4.49 and 4.51 would not).

### Question 37: Fixed-Point / Integer-Cents Financial Strategy

★★★★ HIGH PRIORITY

**Correct Answer: (A) — Integer arithmetic is completely exact and avoids floating-point round-off errors, ensuring accurate penny-perfect accounting.**

STEP-BY-STEP EXPLANATION

- Because integers represent whole discrete counts, integer addition, subtraction, and multiplication are 100% exact in computer hardware (so long as the values stay within bit limits).
- By storing currency in its smallest indivisible unit (cents, pence, or satoshis) as integers, the system completely avoids non-terminating binary fractions like 0.10 and 0.20.
- When displaying amounts to users, the application formats the integer by inserting a decimal point (e.g., 1999 becomes '$19.99').
- This is standard practice across major payment gateways (Stripe, PayPal, banking cores).

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Storing in cents actually increases numerical magnitude by a factor of 100, which requires larger bit widths (e.g., 64-bit integers) to prevent integer overflow.
✗ **(C):** Currency exchange rates still require proportional conversion math regardless of integer or float storage.
✗ **(D):** Financial databases must maintain 100% lossless fidelity; lossy compression is strictly forbidden in financial records.

### Question 38: Floating-Point Architecture (Sign, Exponent, Mantissa)

★★★ MANDATORY

**Correct Answer: (A) — Floating-point numbers can represent an immense range of both extremely large and microscopically small values, but at the cost of potential precision loss and round-off errors.**

STEP-BY-STEP EXPLANATION

- Fixed-width integers have uniform precision (exact to 1 unit) but a restricted numerical range.
- Floating-point format mimics scientific notation ($\pm M \times 2^E$):
- • The exponent allows the binary decimal point to 'float', enabling representation of vast cosmic scales (e.g., $10^{38}$) and subatomic scales (e.g., $10^{-38}$) within 32 or 64 bits.
- • However, the mantissa has a fixed number of precision bits (e.g., 53 bits in 64-bit double), meaning numbers with more significant digits than the mantissa must be rounded off.
- This fundamental architectural trade-off provides massive dynamic range at the expense of absolute precision.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Floating-point numbers can overflow (to Infinity) and underflow (to 0), and allocate an explicit sign bit for negative numbers.
✗ **(C):** Floating-point ALUs are physically more complex and use far more transistors than integer ALUs; they operate entirely locally on-chip.
✗ **(D):** Floating-point numbers have finite precision (e.g., ~15-17 decimal digits in 64-bit double); they cannot provide infinite decimal precision.

### Question 39: Truncation vs. Rounding

★★★ MANDATORY

**Correct Answer: (B) — Calculations develop a systematic downward bias because positive numbers are always rounded down toward zero.**

STEP-BY-STEP EXPLANATION

- Truncation simply discards all bits beyond the register limit without inspecting their values.
- For positive numbers, discarding fractional bits always makes the stored value smaller than the true mathematical value.
- When thousands of calculations are performed, this creates a persistent, cumulative downward statistical bias.
- In contrast, standard IEEE 754 'round-to-nearest, ties-to-even' rounds up half the time and down half the time, canceling out directional bias.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Truncation affects fractional precision, not integer register overflow exceptions.
✗ **(C):** The sign bit is stored separately and is completely unaffected by mantissa bit truncation.
✗ **(D):** Truncation exacerbates rounding error by skewing all positive calculations in one direction, rather than canceling errors out.

### Question 40: Comparison: Integer Overflow vs. Floating-Point Round-Off

★★★★★ VERY IMPORTANT

**Correct Answer: (D) — Phenomenon X is Integer Overflow; Phenomenon Y is Floating-Point Round-Off.**

STEP-BY-STEP EXPLANATION

- Review the distinct attributes of both phenomena defined in AP CSP CED Big Idea 2 (DAT-1.B):
- • **Integer Overflow (Phenomenon X):** Occurs when integer calculations produce an integer too large to fit in fixed-width bits, causing rollover/wraparound or hardware exceptions.
- • **Floating-Point Round-Off (Phenomenon Y):** Occurs when continuous or repeating fractional real numbers are approximated within finite floating-point mantissa bits, causing tiny precision discrepancies.
- Therefore, X is Integer Overflow and Y is Floating-Point Round-Off.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Compression algorithms relate to file encoding and size reduction, not arithmetic register behavior.
✗ **(B):** Syntax errors relate to code grammar; these are mathematical representations in hardware.
✗ **(C):** Inverts the definitions of overflow and round-off.

### Question 41: Continuous Analog vs. Discrete Digital Data

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — Analog data changes smoothly and continuously with an infinite number of possible intermediate states, whereas digital data is broken into discrete, separate values represented by binary bits.**

STEP-BY-STEP EXPLANATION

- College Board Big Idea 2 (DAT-1.C) explicitly defines analog vs digital data:
- • **Analog data:** Continuous values that represent physical phenomena smoothly over time or space (such as sound waves, ambient temperature, light brightness, or mercury rising in a thermometer) with an infinite range of fractional values.
- • **Digital data:** Discrete approximations created by sampling and quantizing continuous data into finite numeric values represented by binary bits (0s and 1s).
- Because computers have finite storage, they must approximate continuous analog reality into discrete digital numbers.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Human ears naturally perceive continuous analog pressure waves in the air, not raw digital bitstreams.
✗ **(B):** Both hard drives and RAM store digital data; storage medium type does not determine whether data is analog or digital.
✗ **(D):** Analog signals degrade significantly through physical interference and generation loss; digital data can be copied infinitely without degradation.

### Question 42: The Digitization Sampling Process

★★★★★ VERY IMPORTANT

**Correct Answer: (A) — The analog audio wave is measured (sampled) at regular time intervals, and each measured voltage is converted into a binary number.**

STEP-BY-STEP EXPLANATION

- Digitizing an analog sound wave requires a two-step sampling and quantization process:
- 1. **Sampling:** An Analog-to-Digital Converter (ADC) measures the electrical voltage from the microphone at precise, fixed time intervals (e.g., 44,100 times per second).
- 2. **Quantization:** Each sampled voltage amplitude is assigned the closest discrete digital value from a finite set of levels determined by the bit depth.
- These numbers are stored as binary sequences representing the audio wave over time.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Physical vibrations cannot be stored directly on silicon SSDs without being digitized into binary bits.
✗ **(C):** JPEG is an image compression format, not an audio conversion mechanism, and compression happens after digitization.
✗ **(D):** Sound waves fluctuate thousands of times per second; measuring once cannot capture a song.

### Question 43: Impact of Sampling Rate on Audio Quality

★★★★★ VERY IMPORTANT

**Correct Answer: (A) — Setting B captures higher audio frequencies and reproduces sound with much higher fidelity, but produces a file that is four times larger.**

STEP-BY-STEP EXPLANATION

- The **sampling rate** is the frequency at which analog signal measurements are captured per second.
- • Setting B samples 44,100 times/sec compared to Setting A's 11,025 times/sec (a 4:1 ratio: 44,100 / 11,025 = 4).
- • Under the Nyquist theorem, a higher sampling rate captures higher audio frequencies, producing richer, more natural, and higher-fidelity audio reproduction.
- • Because four times as many samples are captured every second, the resulting uncompressed file size is exactly 4 times larger.
- This exemplifies the fundamental trade-off between fidelity and file size.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Taking 4 times as many samples increases file size by 4x, rather than making it smaller.
✗ **(C):** Higher sampling rates reduce distortion; they do not alter the audio genre or voice.
✗ **(D):** Dynamic volume range is controlled by bit depth (amplitude resolution), not sampling rate.

### Question 44: Bit Depth (Resolution) and Quantization Levels

★★★★★ VERY IMPORTANT

**Correct Answer: (B) — Bit depth determines the number of bits allocated to store the amplitude of each sample; increasing from 8 to 16 bits increases the number of available amplitude levels from 256 to 65,536, dramatically reducing quantization noise.**

STEP-BY-STEP EXPLANATION

- **Bit depth** (sample resolution) defines the number of binary bits allocated to record the amplitude (volume/height) of each individual sample:
- • With 8 bits per sample: $2^8$ = 256 discrete amplitude levels.
- • With 16 bits per sample: $2^{16}$ = 65,536 discrete amplitude levels.
- Increasing bit depth allows the digital system to measure subtle volume gradations with far greater precision, minimizing the difference between the true analog wave and its discrete digital representation (reducing quantization error).
- Sampling rate governs time resolution; bit depth governs amplitude resolution.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Channel count (mono = 1, stereo = 2, surround = 6) is configured separately from sample bit depth.
✗ **(C):** File duration is limited by overall storage capacity, not sample bit depth.
✗ **(D):** Samples per second is the *sampling rate* (Hz), not the bit depth.

### Question 45: Digital Audio File Size Calculation

★★★★ HIGH PRIORITY

**Correct Answer: (B) — 44,100 × 16 × 2 × 60 bits**

STEP-BY-STEP EXPLANATION

- To find the total bit size of an uncompressed sampled audio recording, multiply all contributing dimensions:
- $$\text{Total Bits} = \text{Sample Rate (samples/sec)} \times \text{Bit Depth (bits/sample)} \times \text{Channels} \times \text{Time (seconds)}$$
- Here:
- • Sampling Rate = 44,100 samples/sec.
- • Bit Depth = 16 bits/sample.
- • Channels = 2 (stereo).
- • Duration = 60 seconds.
- Total bits = 44,100 × 16 × 2 × 60 = 84,672,000 bits (~10.58 MB).

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** The parameters are independent dimensions that multiply together, not add.
✗ **(C):** Bit depth must be multiplied, not divided.
✗ **(D):** Stereo doubles the data, so it must be multiplied by 2, not divided by 2.

### Question 46: Quantization Error Definition

★★★★ HIGH PRIORITY

**Correct Answer: (D) — Quantization error (quantization noise)**

STEP-BY-STEP EXPLANATION

- **Quantization error** is the difference between the true, continuous analog signal value and the discrete digital approximation assigned to it by the Analog-to-Digital Converter.
- Because a digital system has a finite number of quantization levels (governed by its bit depth), continuous analog values falling between two discrete steps must be rounded to the nearest level.
- This rounding deviation manifests as quantization error (or quantization noise).
- Higher bit depths create closer spacing between levels, reducing this error.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Syntax errors occur in programming language grammar.
✗ **(B):** Overflow occurs when a whole number exceeds the maximum capacity of a register, not fractional rounding during conversion.
✗ **(C):** Dictionary compression is a text compression method, not a physical signal conversion artifact.

### Question 47: Image Digitization (Spatial Sampling & Color Depth)

★★★★ HIGH PRIORITY

**Correct Answer: (A) — The continuous 2D painting is divided into a discrete grid of tiny picture elements (pixels), and each pixel's color is measured and quantized into binary values (such as 24-bit RGB).**

STEP-BY-STEP EXPLANATION

- Digitizing visual images parallels digital audio sampling:
- • **Spatial Sampling:** The continuous physical surface is sampled across space by dividing it into a regular 2D grid of discrete dots or picture elements called **pixels** (measured in DPI / resolution).
- • **Color Quantization:** The continuous color and brightness at each pixel is measured by optical sensors and quantized into discrete binary values using a specific color bit depth (e.g., 24-bit RGB with 8 bits each for red, green, and blue).
- Higher pixel density (resolution) and higher color bit depth yield a more faithful digital representation.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Image sensors capture light intensities across a 2D pixel grid, not linguistic descriptions of brush strokes.
✗ **(C):** Scanners produce 2D image matrices of color bits, not sound waveforms.
✗ **(D):** Color scanners measure reflected light frequencies, not physical paint mass.

### Question 48: Trade-offs in Sampling Precision and Storage

★★★★★ VERY IMPORTANT

**Correct Answer: (B) — Higher sampling rates and bit depths capture greater acoustic detail for accurate bird identification, but produce larger files that take longer to transmit and consume more storage.**

STEP-BY-STEP EXPLANATION

- College Board Big Idea 2 (DAT-1.C) emphasizes evaluating trade-offs in digitization:
- • Increasing sampling rate and bit depth improves fidelity, enabling wildlife experts to distinguish subtle acoustic frequencies and calls.
- • However, higher sampling parameters directly increase the total bits per second, resulting in larger file sizes, increased mobile bandwidth consumption, and faster storage depletion.
- The rangers must select the minimum sampling threshold that captures species call frequencies without overwhelming their low-bandwidth network.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Higher sampling rates generate *more* bits and larger files, not smaller files.
✗ **(C):** Lower sampling rates reduce the maximum captured frequency (Nyquist limit), not increase it.
✗ **(D):** No digital recording can achieve infinite precision, and all files consume non-zero storage space.

### Question 49: Digital Exact Reproduction vs. Analog Generation Loss

★★★ MANDATORY

**Correct Answer: (B) — Digital copies simply duplicate exact discrete binary bit values (0s and 1s), preventing physical signal noise from corrupting the underlying data.**

STEP-BY-STEP EXPLANATION

- Analog media (cassette tapes, vinyl records) store signals as continuous physical modulations (magnetic particle alignments, vinyl groove depths).
- Every time an analog copy is made, physical mechanical and electromagnetic imperfections introduce random noise that becomes permanent (generation loss).
- Digital media stores data as discrete binary numbers (0s and 1s).
- During file transfer, error-checking protocols verify that every bit transferred matches the source bit identically. As long as a 1 remains a 1 and a 0 remains a 0, the copy is a bit-for-bit perfect clone.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Digital files can be copied indefinitely without restriction (unless restricted by DRM software).
✗ **(C):** Flash drives operate at room temperature; temperature is not the reason digital bits copy faithfully.
✗ **(D):** Digital computers use silicon transistors, not vacuum tubes.

### Question 50: Classifying Real-World Systems as Analog or Digital

★★★★ HIGH PRIORITY

**Correct Answer: (C) — Items 1 and 3 are analog; Items 2 and 4 are digital.**

STEP-BY-STEP EXPLANATION

- • **Item 1 (Mercury thermometer):** Analog. Mercury expands continuously and smoothly based on heat; there are no discrete numeric steps.
- • **Item 2 (Digital scale):** Digital. A sensor converts physical pressure into discrete numbers displayed in specific units (e.g., 0.1 gram steps).
- • **Item 3 (Vinyl record):** Analog. The continuous physical shape of the groove mechanically drives the needle to recreate continuous sound waves.
- • **Item 4 (E-book reader):** Digital. Text and images are represented by discrete matrices of binary pixels on an electronic ink display.
- Therefore, 1 and 3 are analog, while 2 and 4 are digital.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** The digital scale displays discrete numbers, making it digital, not analog.
✗ **(B):** Measuring physical quantities does not make an instrument digital; continuous physical gauges are analog.
✗ **(D):** A vinyl record is a classic continuous analog recording medium.

### Question 51: Definition of Lossless Compression

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — A compression algorithm that reduces file size while allowing the exact original data to be perfectly reconstructed bit-for-bit upon decompression.**

STEP-BY-STEP EXPLANATION

- College Board Big Idea 2 (DAT-1.D) defines lossless compression as a process that reduces the number of bits required to store or transmit data while guaranteeing that the decompressed data is completely identical to the original uncompressed data.
- No information is lost or discarded during compression or decompression.
- Common lossless algorithms include Run-Length Encoding (RLE), Huffman coding, and Lempel-Ziv-Welch (LZW), which are used in ZIP archives, PNG images, and FLAC audio.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Compression algorithms operate purely on digital binary data; they do not convert files into analog sound waves.
✗ **(B):** Lossless compression operates on any digital data (text, images, audio, binaries), not exclusively encrypted files.
✗ **(D):** Permanently discarding data describes **lossy compression**, not lossless.

### Question 52: Scenarios Requiring Lossless Compression

★★★★★ VERY IMPORTANT

**Correct Answer: (D) — A compiled software executable (.exe) or computer source code file, where losing or altering a single bit would corrupt program execution.**

STEP-BY-STEP EXPLANATION

- Lossless compression is mandatory whenever perfect data fidelity is non-negotiable:
- • **Source code and executable programs:** A single changed or omitted bit can alter an opcode, flip a condition, or trigger a fatal crash.
- • **Text documents, financial records, and medical files:** Altering characters or numerical values corrupts meaning and integrity.
- In contrast, video streaming, casual social media photos, and background music can tolerate lossy compression because human perception will not notice minor dropped nuances.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Background restaurant music uses lossy MP3/AAC compression.
✗ **(B):** Streaming video uses lossy compression (like H.264, HEVC, AV1) to minimize cellular bandwidth.
✗ **(C):** Social media photos use lossy JPEG compression to reduce storage and transmission costs.

### Question 53: Run-Length Encoding (RLE) Mechanics

★★★★★ VERY IMPORTANT

**Correct Answer: (D) — `8W 3B 5W`**

STEP-BY-STEP EXPLANATION

- Run-Length Encoding (RLE) compresses data by replacing consecutive runs of identical symbols with a run count followed by the symbol:
- • First run: Eight 'W's in a row -> `8W`.
- • Second run: Three 'B's in a row -> `3B`.
- • Third run: Five 'W's in a row -> `5W`.
- Combining the runs produces `8W 3B 5W`.
- This reduces the 16 individual characters into 3 concise run-length pairs without losing any pixel data.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Transposes the counts: there are 3 black pixels and 5 white pixels, not 5 black and 3 white.
✗ **(B):** `8W 3B 5W` is the standard run-length format (count followed by symbol).
✗ **(C):** Incorrectly sums all white pixels together without preserving their spatial order.

### Question 54: Limitations of Run-Length Encoding (RLE)

★★★★ HIGH PRIORITY

**Correct Answer: (B) — When the data consists of rapidly alternating individual symbols with no consecutive repetitions (e.g., `WBWBWBWBWB`).**

STEP-BY-STEP EXPLANATION

- RLE relies on consecutive repetitions of the same value (long 'runs') to achieve compression.
- If a sequence has no consecutive repetitions (e.g., `WBWBWBWBWB` with 10 characters):
- • In RLE, each symbol has a run length of 1: `1W1B1W1B1W1B1W1B1W1B`.
- • This output requires 20 characters (a count digit plus the letter for each item) to represent an original 10-character string.
- The compressed data is twice as large as the original uncompressed data.
- This demonstrates that RLE is inefficient for noisy, high-entropy, or rapidly fluctuating data.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** A run of 1,000 identical letters compresses dramatically from 1,000 characters to `1000A` (5 characters).
✗ **(C):** Large solid blocks of color provide long consecutive runs (e.g., `500W`), which RLE compresses with massive efficiency.
✗ **(D):** Hexadecimal format has no bearing on RLE effectiveness; compression depends strictly on data repetition.

### Question 55: Dictionary-Based Lossless Compression Mechanics

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — It replaces lengthy, frequently repeated patterns of text with short numeric tokens, storing the dictionary alongside the encoded tokens.**

**STEP-BY-STEP EXPLANATION**

- Dictionary-based algorithms (like LZW used in ZIP and GIF) scan data for recurring substrings, phrases, or patterns.
- When a repeated sequence is detected, it is added to a lookup table (dictionary) and assigned a compact numerical token or index.
- Subsequent occurrences of that entire phrase in the document are replaced by the short token.
- To decompress the file, the decoder reads the token, looks up the corresponding phrase in the dictionary, and reconstructs the exact original text bit-for-bit.
- This is completely lossless.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** Text files are stored as structured character strings, not converted into lossy JPEG images.
✗ **(B):** Deleting vowels permanently discards data, which would be lossy compression and corrupt the text.
✗ **(D):** Dictionary compression uses structured indexed tables, not random punctuation substitution.

### Question 56: Decoding Dictionary-Compressed Data

★★★★ HIGH PRIORITY

**Correct Answer: (C) — "the blue sky the clear sky"**

**STEP-BY-STEP EXPLANATION**

- To decompress, replace each token in the encoded sequence with its corresponding dictionary entry in order:
- • `4` -> "the"
- • `1` -> "blue"
- • `2` -> "sky"
- • `4` -> "the"
- • `3` -> "clear"
- • `2` -> "sky"
- Combining these words in sequence yields: "the blue sky the clear sky".

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** Omits words and misaligns the token order.
✗ **(B):** Starts with token 1 ("blue") instead of token 4 ("the").
✗ **(D):** Reverses the positions of "blue" and "sky".

### Question 57: Compression Ratio and Percentage Savings Calculation

★★★★ HIGH PRIORITY

**Correct Answer: (A) — Compression ratio of 4:1 (75% space reduction)**

**STEP-BY-STEP EXPLANATION**

- Compute the compression metrics:
- 1. **Compression Ratio:**
- $$\text{Ratio} = \frac{\text{Uncompressed Size}}{\text{Compressed Size}} = \frac{400\text{ MB}}{100\text{ MB}} = \frac{4}{1} = 4:1$$
- 2. **Percentage Reduction (Space Saved):**
- $$\text{Space Saved} = \frac{\text{Original} - \text{Compressed}}{\text{Original}} \times 100\% = \frac{400 - 100}{400} \times 100\% = \frac{300}{400} \times 100\% = 75\%$$
- Therefore, the compression ratio is 4:1 and the space reduction is 75%.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(B):** The ratio is defined as uncompressed / compressed (4:1), and space reduction cannot exceed 100%.
✗ **(C):** 25% is the percentage of the file remaining (100 / 400), not the percentage of space saved (reduced).
✗ **(D):** 2:1 would result in a compressed size of 200 MB, not 100 MB.

### Question 58: Common Lossless Image and Audio Formats

★★★ MANDATORY

**Correct Answer: (D) — PNG, GIF, and FLAC**

STEP-BY-STEP EXPLANATION

- Review the compression categories of common multimedia formats:
- • **Lossless formats:**
- - **PNG (Portable Network Graphics):** Lossless image format ideal for graphics, line art, and transparent backgrounds.
- - **GIF (Graphics Interchange Format):** Lossless image format (using LZW) supporting 256 colors and simple animations.
- - **FLAC (Free Lossless Audio Codec):** Lossless audio format preserving 100% of master recording fidelity.
- • **Lossy formats:** JPEG (lossy image), MP3 (lossy audio), MP4/AAC (lossy video/audio).
- Therefore, Option A contains exclusively lossless formats.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** JPEG, MP3, and MP4 are all classic lossy compression formats.
✗ **(B):** MP4 uses lossy video compression.
✗ **(C):** JPEG and MP3 are lossy formats.

### Question 59: Theoretical Limits of Lossless Compression (Pigeonhole Principle)

★★★★ HIGH PRIORITY

**Correct Answer: (D) — By the Pigeonhole Principle, there are fewer short bit-sequences than long bit-sequences; if every file were compressed to a shorter length, multiple distinct original files would map to the identical compressed sequence, making 100% lossless decompression impossible.**

STEP-BY-STEP EXPLANATION

- Consider files of length N bits: there are $2^N$ possible files.
- The total number of all possible files with length strictly less than N bits is:
- $2^0 + 2^1 + 2^2 + \dots + 2^{N-1} = 2^N - 1$.
- Notice that $2^N - 1$ is strictly less than $2^N$.
- By the **Pigeonhole Principle**, you cannot map $2^N$ distinct inputs uniquely to only $2^N - 1$ outputs without at least two inputs producing the exact same compressed output.
- If two different original files produce the same compressed file, the algorithm cannot know which one to reconstruct during decompression.
- Therefore, any lossless compression algorithm that makes some files smaller must inevitably make other files (such as random or already-compressed data) larger or the same size.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Operating systems and memory chips handle files of any size, including 0 bytes.
✗ **(B):** There are no legal bans; this is a mathematical theorem of information theory proved by Claude Shannon.
✗ **(C):** Compression works on all binary files regardless of whether lengths are prime or composite.

### Question 60: Comparing RLE vs. Dictionary-Based Compression

★★★★ HIGH PRIORITY

**Correct Answer: (C) — Algorithm 1 (RLE) is preferred when the dataset contains long contiguous runs of identical repeated values (such as simple icon graphics with solid backgrounds), whereas Algorithm 2 is superior for natural language text with repeated words separated by other characters.**

STEP-BY-STEP EXPLANATION

- • **RLE (Algorithm 1)** is computationally lightweight and excels when identical values appear in immediate consecutive sequence (e.g., solid color pixel rows: 200 white pixels followed by 200 black pixels).
- • **Dictionary compression (Algorithm 2)** excels when patterns or words recur throughout the document but are separated by other words (e.g., the word 'computational' appearing 50 times across an essay). RLE cannot compress separated words, but dictionary compression handles them effortlessly.
- Both algorithms are 100% lossless.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Both RLE and dictionary compression are 100% lossless; neither discards data.
✗ **(B):** 4K video requires lossy compression (HEVC/AV1), not simple RLE.
✗ **(D):** Neither lossless algorithm can compress truly random noise.

### Question 61: Definition and Core Mechanism of Lossy Compression

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — A compression process that achieves significant file size reduction by permanently discarding data that is deemed redundant or imperceptible to human senses, making the original data impossible to reconstruct bit-for-bit.**

**STEP-BY-STEP EXPLANATION**

- College Board Big Idea 2 (DAT-1.D) defines lossy compression:
- • It significantly reduces file size by permanently discarding less important or redundant details.
- • Because data is discarded, the original uncompressed file can **never** be reconstructed exactly bit-for-bit upon decompression.
- • It exploits human perceptual limitations (in vision and hearing) to remove information that humans are unlikely to notice, achieving compression ratios of 10:1 to 20:1 or higher (e.g., JPEG, MP3, MP4).

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** Lossy compression is an intentional mathematical algorithm, not a hardware defect or memory glitch.
✗ **(B):** Temporarily hiding data while preserving exact reconstruction describes lossless compression or archiving, not lossy.
✗ **(D):** Lossy compression is used almost exclusively for multimedia (images, audio, video), never for standard text where discarded words would corrupt meaning.

### Question 62: Perceptual Encoding Principles (Psychoacoustics & Psychovisuals)

★★★★★ VERY IMPORTANT

**Correct Answer: (B) — Human sensory organs have biological thresholds: the human eye is far more sensitive to changes in brightness (luminance) than subtle color shifts (chrominance), and human ears cannot perceive extremely high frequencies or sounds drowned out by louder nearby frequencies.**

**STEP-BY-STEP EXPLANATION**

- Perceptual encoding exploits biological human perception limits:
- • **Psychovisuals (JPEG):** Human retinas have far more rod cells (detecting brightness/luminance) than cone cells (detecting color/chrominance). JPEG aggressively discards subtle color data (chroma subsampling) while preserving luminance, so the human brain barely perceives the loss.
- • **Psychoacoustics (MP3):** Human hearing cannot detect frequencies beyond ~20 kHz, nor quiet sounds that play at the exact same moment as loud sounds (auditory masking). MP3 strips out these inaudible sound components completely.
- Discarding imperceptible details yields massive storage savings with high subjective quality.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** Human brains process optical neural signals; they do not manufacture physical digital RAM.
✗ **(C):** Human sensory acuity varies across individuals; perceptual codecs target general human biological averages.
✗ **(D):** The speed of light is a physical constant; software algorithms cannot accelerate photons.

### Question 63: Irreversibility of Lossy Compression

★★★★★ VERY IMPORTANT

**Correct Answer: (B) — No, because lossy compression permanently discards image data; once removed, the original bits cannot be recovered or reconstructed.**

**STEP-BY-STEP EXPLANATION**

- Lossy compression is fundamentally **irreversible**.
- When the JPEG encoder compressed the image from 50 MB to 3 MB, it permanently discarded fine color gradients, subtle high-frequency edge textures, and high-precision sensor data.
- That discarded information no longer exists anywhere on storage media.
- Decompressing a JPEG simply renders the remaining approximated data on screen; it cannot magically invent or restore the original discarded sensor bits.
- Only lossless compression allows 100% bit-for-bit reversal.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** Renaming a file extension only alters how operating systems associate icons; it does not reconstruct missing binary data.
✗ **(C):** Motherboards do not store hidden analog backups of user photo files.
✗ **(D):** Unzipping software only functions on lossless archives (like ZIP); it cannot restore discarded lossy image bits.

### Question 64: Lossy Compression Artifacts

★★★★ HIGH PRIORITY

**Correct Answer: (D) — Compression artifacts**

STEP-BY-STEP EXPLANATION

- **Compression artifacts** are noticeable, unintended visual or auditory defects introduced into digital media when lossy compression algorithms discard too much data.
- • In JPEG, the Discrete Cosine Transform (DCT) operates on 8×8 pixel blocks. When high-frequency coefficients are heavily quantized, the boundaries between 8×8 blocks become visible ('macroblocking'), and sharp edges develop fuzzy halos ('ringing').
- • In audio (MP3), aggressive lossy compression produces watery, metallic swooshing artifacts or muffled cymbals.
- These artifacts demonstrate the quality trade-off when pushing lossy compression too far.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Syntax parsing errors occur in code text compilers.
✗ **(B):** Quantization error occurs during the initial analog-to-digital conversion, whereas compression artifacts arise during secondary lossy encoding.
✗ **(C):** Integer overflow causes counters to roll over to 0 or crash, not 8×8 visual pixel blockiness.

### Question 65: Format Selection: PNG vs. JPEG for Logos and Text

★★★★★ VERY IMPORTANT

**Correct Answer: (D) — PNG, because its lossless compression preserves crisp, sharp edges without ringing artifacts and natively supports transparent alpha channels.**

STEP-BY-STEP EXPLANATION

- Choosing between PNG (lossless) and JPEG (lossy) depends on image content:
- • **PNG (Lossless):** Ideal for graphics, logos, illustrations, diagrams, screenshots, and text. It preserves 100% sharp pixel contrasts without blurry halos and supports transparent backgrounds (alpha channels).
- • **JPEG (Lossy):** Designed for continuous-tone photographic scenery with complex natural color gradations. When applied to sharp text or high-contrast logos, JPEG creates blurry ringing artifacts around letter edges and does not support transparency.
- Therefore, PNG is the correct choice.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** MP3 is an audio format, not an image format.
✗ **(B):** JPEG handles sharp text poorly, creating fuzzy artifact halos, and does not support transparency.
✗ **(C):** All modern browsers support PNG, JPEG, SVG, and WebP, not just GIF.

### Question 66: Generation Loss Through Repeated Lossy Saving

★★★★ HIGH PRIORITY

**Correct Answer: (C) — Generation loss: each time a file is re-encoded with lossy compression, the algorithm discards another layer of data and amplifies existing compression artifacts.**

STEP-BY-STEP EXPLANATION

- **Generation loss** occurs when lossy compression is applied repeatedly to the same content across multiple generations (cycles) of editing and saving.
- Each time the file is saved as a JPEG, the encoder quantizes and discards more frequency data, creating new compression artifacts while compounding existing ones.
- Over multiple cycles, the image accumulates severe blockiness, blurriness, and color distortion.
- In contrast, re-saving a lossless file (such as PNG or TIFF) causes zero quality loss regardless of how many times it is opened and saved.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Email protocols transfer binary attachments bit-for-bit without automatically cutting pixel dimensions in half.
✗ **(B):** Image quality degradation is caused by lossy DCT quantization, not floating-point rounding of metadata.
✗ **(D):** Bits do not 'run out'; generation loss is caused by compounding algorithmic quantization.

### Question 67: Comparison of Compression Ratios: Lossless vs. Lossy

★★★★ HIGH PRIORITY

**Correct Answer: (B) — Lossless algorithms typically achieve modest ratios of around 2:1 to 3:1 (50% to 66% reduction), whereas lossy algorithms can routinely achieve 10:1 to 20:1 or greater (90% to 95% reduction).**

**STEP-BY-STEP EXPLANATION**

- Because lossless compression must guarantee 100% bit-for-bit reversibility, it is constrained by theoretical entropy limits (Shannon entropy):
- • **Lossless compression (ZIP, PNG, FLAC):** Typically achieves ratios between 1.5:1 and 3:1 (reducing files by roughly 30% to 70%).
- • **Lossy compression (JPEG, MP3, MP4):** Because it can permanently discard non-essential data, it easily achieves ratios of 10:1 (90% space saved), 20:1 (95% space saved), or even 50:1 for streaming video.
- This massive size reduction is the primary justification for accepting lossy degradation in consumer media.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** Lossless algorithms cannot achieve 100:1 on complex data; lossy achieves far higher ratios than lossless.

✗ **(C):** Lossless compression makes redundant files significantly smaller, not larger.

✗ **(D):** Ratios differ dramatically based on whether data can be permanently discarded.

### Question 68: Identifying Common Lossy Multimedia Formats

★★★ MANDATORY

**Correct Answer: (B) — JPEG, MP3, and MP4**

**STEP-BY-STEP EXPLANATION**

- • **JPEG (Joint Photographic Experts Group):** Lossy compression standard for digital photographs.
- • **MP3 (MPEG Audio Layer III):** Lossy compression standard for audio recordings.
- • **MP4 (MPEG-4 Part 14):** Lossy container format widely used for streaming digital video and audio (utilizing lossy video codecs like H.264/AAC).
- All three formats permanently discard data to achieve small file sizes, making Option A the correct group.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** PNG and FLAC are lossless formats.

✗ **(C):** PNG, FLAC, and ZIP are all strictly lossless formats.

✗ **(D):** BMP, WAV, and TIFF are standard uncompressed or lossless formats.

### Question 69: Evaluating Suitability of Lossy Compression in Healthcare

★★★★★ VERY IMPORTANT

**Correct Answer: (A) — Lossy compression permanently discards fine detail and introduces subtle pixel artifacts that could obscure an early-stage tumor or simulate a false micro-fracture, leading to fatal misdiagnosis.**

**STEP-BY-STEP EXPLANATION**

- Medical diagnostic imaging demands absolute clinical fidelity.
- Early-stage malignancies, microscopic fractures, and subtle tissue lesions often appear as minute, faint clusters of a few pixels.
- Lossy compression algorithms (like JPEG) discard subtle high-frequency gradients and create blocky artifacts that can either erase an actual micro-calcification or create phantom artifacts that mimic disease.
- To prevent medical misdiagnosis and legal liability, standard medical imaging protocols (DICOM) mandate uncompressed or strictly lossless compression (such as JPEG-LS or Lossless PNG) for diagnostic scans.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(B):** JPEG and MP4 open on all operating systems and desktop monitors.

✗ **(C):** Patient metadata in medical DICOM headers is managed by database access controls, not the pixel compression codec.

✗ **(D):** Hard drives store arbitrary binary bits regardless of file format; hospital computers easily support JPEG.

### Question 70: Comprehensive Compression Decision Matrix

★★★★ HIGH PRIORITY

**Correct Answer: (D) — Projects 1 and 3 should use Lossy; Projects 2 and 4 must use Lossless.**

STEP-BY-STEP EXPLANATION

- Evaluate the tolerance for data loss across each project:
- • **Project 1 (Game textures):** Visual graphics can tolerate lossy compression (like DXT/ETC texture compression) to meet mobile download limits.
- • **Project 2 (Tax database):** Financial data requires 100% exact numerical fidelity; must be **lossless**.
- • **Project 3 (Satellite voice):** Voice transmission over narrow bandwidth prioritizes intelligibility and small footprint; uses **lossy** speech codecs (e.g., AMR, Opus).
- • **Project 4 (OS patch):** Executable binary code must be 100% exact; must be **lossless**.
- Thus, Projects 1 & 3 use Lossy, while Projects 2 & 4 must use Lossless.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Project 2 requires lossless fidelity.
✗ **(B):** Project 2 is financial data and can never use lossy compression.
✗ **(C):** Cellular networks and mobile storage have strict finite limits; storage and bandwidth are never free.

### Question 71: Definition and Role of Metadata

★★★★★ VERY IMPORTANT

**Correct Answer: (D) — Data that provides descriptive information about other data, such as creation timestamps, author identity, file size, or resolution.**

STEP-BY-STEP EXPLANATION

- College Board Big Idea 2 (DAT-2.A) defines metadata simply and strictly as 'data about data'.
- Metadata does not represent the primary content of the file itself (such as the actual paragraph of text or the actual visual colors of a photograph).
- Instead, metadata provides structured context describing the data, such as:
- • Date and time of creation or modification.
- • Author or device that created the file.
- • File size, format, dimensions, or duration.
- • GPS geolocation coordinates.
- Metadata helps systems organize, search, filter, and process data effectively.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Metadata is standard structural information, not malicious malware or a virus.
✗ **(B):** Physical voltage belongs to physical hardware/networking (Layer 1), not descriptive data representation.
✗ **(C):** Compiled machine code consists of program instructions (opcodes), not descriptive data attributes.

### Question 72: Effect of Modifying Metadata on Primary Content

★★★★★ VERY IMPORTANT

**Correct Answer: (B) — The visual pixel colors and resolution of the photograph remain completely unchanged because metadata is stored separately from the primary image data.**

STEP-BY-STEP EXPLANATION

- A core principle in AP CSP (DAT-2.A.2) is that changing metadata does not modify the primary content of the underlying data.
- File formats store metadata in a dedicated header block (such as an EXIF or ID3 tag section) distinct from the payload containing the raw or compressed pixel array.
- Editing tags, author names, or timestamps updates only the header bits; the image pixels themselves are never touched or re-compressed.
- Therefore, the visual image remains 100% identical.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Metadata changes have zero effect on image pixel dimensions or resolution.
✗ **(C):** Editing metadata does not trigger lossy re-encoding; no pixels are blurred.
✗ **(D):** The file remains an image; changing tags cannot convert graphics into audio.

### Question 73: Utility of Metadata in Search, Indexing, and Organization

★★★★ HIGH PRIORITY

**Correct Answer: (B) — The database indexes structured metadata fields (such as genre tags and release year) without needing to decompress or analyze the audio waveforms of the songs.**

STEP-BY-STEP EXPLANATION

- Metadata enables efficient indexing, querying, and sorting of massive datasets.
- Instead of expending massive computational power decompressing and acoustically analyzing hundreds of gigabytes of raw audio waveforms, the database simply queries indexed metadata fields: `WHERE genre = 'Jazz' AND release_year = 1959`.
- Because text/integer metadata is tiny and indexed in B-trees or hash tables, the search executes in milliseconds.
- This exemplifies the computational efficiency that structured metadata provides.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Musicians record audio; metadata is generated and cataloged by software systems.
✗ **(C):** Audio analysis of 80 million songs per search would take hours and crash servers; databases query metadata tags instead.
✗ **(D):** Searching a database queries records; it does not delete songs from the catalog.

### Question 74: Smartphone Photo EXIF Metadata Components

★★★★★ VERY IMPORTANT

**Correct Answer: (A) — Date and time taken, GPS latitude and longitude coordinates, camera device model, and exposure/ISO settings.**

STEP-BY-STEP EXPLANATION

- Standard EXIF metadata embedded by digital cameras and smartphones includes:
- • Date, hour, minute, and second the photo was captured.
- • GPS geolocation data (latitude, longitude, altitude) if location permissions are enabled.
- • Camera hardware details (phone manufacturer, model, lens aperture, focal length).
- • Exposure settings (shutter speed, ISO sensitivity, flash status).
- • Image properties (pixel width, pixel height, color space).
- This descriptive context travels with the image file wherever it is sent.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Cameras only record data captured during the shutter exposure, not continuous prior 24-hour video.
✗ **(C):** Cameras do not have access to or embed personal banking passwords or social security numbers in image files.
✗ **(D):** Battery chemistry is a manufacturing attribute, not optical camera metadata.

### Question 75: Privacy and Security Risks of Metadata

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — The student failed to strip the EXIF metadata from the photo, allowing the stranger to read the embedded GPS latitude and longitude coordinates.**

STEP-BY-STEP EXPLANATION

- College Board Big Idea 2 (DAT-2.B) highlights the privacy risks posed by automated metadata collection.
- Smartphones automatically record precise GPS coordinates in a photo's EXIF metadata header unless location tagging is disabled.
- If the raw image file is uploaded without 'scrubbing' (stripping) this metadata, anyone who downloads or inspects the file can extract the exact geographic coordinates (within a few meters) and look them up on a mapping service to reveal the student's home address.
- This illustrates why social media platforms automatically strip EXIF metadata from public image uploads to protect user privacy.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Converting pixels to ASCII does not read human thoughts.
✗ **(B):** While high-resolution reflections are a theoretical forensic trope, extracting embedded GPS metadata tags is the standard, effortless method used.
✗ **(D):** Lossy compression discards subtle pixel details; it never fabricates street address text.

### Question 76: Metadata in Network Communications (Packet Headers)

★★★★ HIGH PRIORITY

**Correct Answer: (D) — The payload is the actual content of the email text being delivered, while the metadata (packet header) contains routing information like source IP address, destination IP address, and packet sequence numbers.**

**STEP-BY-STEP EXPLANATION**

- In networking architecture (and data representation):
- • **Payload (Primary Data):** The actual user information being transmitted (e.g., the text of the email or chunks of an image file).
- • **Metadata (Header):** Data *about* the transmission that enables internet protocols to function, including the sender's IP address, the recipient's IP address, packet sequence order (for TCP reconstruction), packet size, and checksums for error detection.
- Routers inspect only the metadata headers to guide packets across intermediate networks without reading the private email payload.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** Inverts the terms: the message body is the payload, not the metadata.
✗ **(B):** Routers forward the payload intact; discarding the payload would destroy the message.
✗ **(C):** Both headers and payloads are transmitted as binary bits.

### Question 77: Metadata Scrubbing / Sanitization

★★★★ HIGH PRIORITY

**Correct Answer: (D) — Metadata scrubbing (sanitization)**

**STEP-BY-STEP EXPLANATION**

- **Metadata scrubbing** (or data sanitization) is the deliberate process of stripping out sensitive, identifying, or hidden metadata from files before public release or distribution.
- Office documents and PDFs frequently retain hidden metadata, such as:
- • Names and usernames of past editors.
- • Hidden revision comments and deleted text history.
- • Internal file server directory paths (which expose corporate network structure).
- Removing these metadata tags prevents inadvertent intelligence leakage and privacy violations.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(A):** Integer overflow mitigation involves using wider bit registers, not stripping document tags.
✗ **(B):** Digital-to-analog conversion turns binary numbers into physical electrical waves (e.g., in speakers).
✗ **(C):** Lossy perceptual sampling refers to audio/image compression (like MP3/JPEG).

### Question 78: Classifying Data as Primary Content vs. Metadata

★★★ MANDATORY

**Correct Answer: (A) — Element 1 is the primary data; Elements 2, 3, 4, and 5 are metadata.**

**STEP-BY-STEP EXPLANATION**

- • **Primary Content (Payload):** Element 1 ('Please review the attached project proposal.') is the actual human message that Alice communicated to Bob.
- • **Metadata:** Elements 2 (timestamp), 3 (sender address), 4 (recipient address), and 5 (file size) are all descriptive attributes that provide operational, temporal, and routing context *about* the email message without being the message itself.
- Therefore, Element 1 is primary data, and 2, 3, 4, and 5 are metadata.

**WHY OTHER OPTIONS ARE INCORRECT**

✗ **(B):** The core human message is the primary payload, not metadata.
✗ **(C):** Timestamps and file sizes are classic metadata attributes, not message content.
✗ **(D):** File size is a measurement of the email; it is not the message itself.

### Question 79: Big Data Insights Derived Exclusively from Metadata

★★★★★ VERY IMPORTANT

**Correct Answer: (A) — Social networks, daily movement patterns, relationships, and organizational hierarchies can be mapped with high accuracy, even without hearing the contents of any conversation.**

STEP-BY-STEP EXPLANATION

- A critical theme in AP CSP (DAT-2.B) is that **metadata alone is extraordinarily revealing**.
- Even without listening to a single word of audio:
- • Call frequency and duration reveal intimate relationships, family ties, and business partnerships.
- • Timestamps reveal work schedules, sleep habits, and lifestyle routines.
- • Cell tower locations track geographic movements across the city in real time.
- • Calling patterns (e.g., calls to specialized medical clinics or crisis hotlines) can reveal personal health or legal crises.
- This demonstrates why metadata collection involves profound privacy and civil liberties implications.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Phone network cell towers measure radio frequencies, not physiological vitals.
✗ **(C):** Metadata is immensely rich and structured; it provides comprehensive behavioral and relational insight.
✗ **(D):** Speech audio was never recorded; acoustic waveforms cannot be hallucinated from numerical timestamps and phone numbers.

### Question 80: Audio File ID3 Metadata Tags

★★★ MANDATORY

**Correct Answer: (C) — The ID3 metadata header is updated with the new text string, while the acoustic audio waveform data remains completely unaffected.**

STEP-BY-STEP EXPLANATION

- ID3 tags are metadata headers appended to the beginning or end of an MP3 audio file.
- Editing the artist name rewrites a few ASCII/Unicode characters in the metadata tag block.
- The compressed audio payload (the perceptual frequency data representing the music) is completely separate in the file and is not re-encoded, altered, or degraded.
- The audio sounds exactly the same before and after the tag edit.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Editing tags in an MP3 does not magically restore discarded audio frequencies or convert the file to lossless FLAC.
✗ **(B):** Metadata edits only affect digital text fields, requiring zero musical re-recording.
✗ **(D):** Audio playback tempo is determined by sample rate and frame headers, not text strings in ID3 tags.

### Question 81: Transforming Raw Data into Information

★★★★★ VERY IMPORTANT

**Correct Answer: (B) — Raw data consists of unorganized, raw facts, measurements, or symbols; information is produced when that data is processed, structured, and contextualized to reveal patterns or answer questions.**

STEP-BY-STEP EXPLANATION

- College Board Big Idea 2 (DAT-2.A) draws a fundamental distinction between data and information:
- • **Data:** Raw, unprocessed values, sensor readings, numbers, or text strings without inherent meaning on their own (e.g., a list of numbers: [72, 68, 75, 71, 98]).
- • **Information:** The result of analyzing, categorizing, aggregating, or visualizing that data in a meaningful context (e.g., 'The average classroom temperature was 71°F, with one extreme anomalous spike of 98°F when the heater malfunctioned').
- Processing and contextualizing raw data transforms it into actionable human information.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Processing flawed or biased raw data can easily produce biased, inaccurate information ('garbage in, garbage out').
✗ **(C):** Both data and information are viewed on digital screens and computer monitors every day.
✗ **(D):** Data storage format (lossy vs lossless) is independent of whether data has been processed into information.

### Question 82: The Data Cleaning (Scrubbing) Process

★★★★★ VERY IMPORTANT

**Correct Answer: (B) — Data cleaning (data cleansing / data scrubbing)**

STEP-BY-STEP EXPLANATION

- **Data cleaning** is the vital preparatory process of inspecting, standardizing, and repairing raw data before analysis.
- Typical data cleaning steps include:
- • Standardizing date and address formats to uniform schemas.
- • Removing exact duplicate entries caused by accidental re-submissions.
- • Filtering out or correcting impossible outlier values (like negative ages).
- • Handling missing fields (either by removing incomplete records or imputing values).
- Clean data ensures that subsequent statistical analyses, visualizations, and conclusions are valid.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Round-off estimation relates to binary floating-point precision, not fixing survey entry errors.
✗ **(C):** Sampling converts continuous waves into discrete numbers.
✗ **(D):** Run-length encoding is a lossless compression algorithm, not a dataset cleaning process.

### Question 83: Correlation vs. Causation in Data Science

★★★★★ VERY IMPORTANT

**Correct Answer: (D) — Ice cream sales and sunburn cases correlate strongly because both are influenced by a common confounding variable (hot, sunny summer weather); eating ice cream does not cause sunburns.**

STEP-BY-STEP EXPLANATION

- A core principle in AP CSP (DAT-2.C) is that **correlation does not imply causation**.
- • **Correlation:** Indicates that two variables move together statistically (as variable X increases, variable Y also increases).
- • **Causation:** Proves that an increase in variable X directly causes the increase in variable Y.
- Here, both ice cream consumption and sunburn incidents increase in the summer due to a third confounding variable: warm temperatures and prolonged outdoor sunlight exposure.
- Falsely attributing direct causality without experimental evidence is a major analytical fallacy.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** The correlation is statistically real in the data, not a computer overflow bug.
✗ **(B):** Sunburns do not biologically cause cravings; both are correlated through summer weather.
✗ **(C):** Ice cream consumption has no physiological causal mechanism that produces sunburns.

### Question 84: Sampling Bias and Limitations of Datasets

★★★★★ VERY IMPORTANT

**Correct Answer: (A) — Selection (sampling) bias: the survey sampled only individuals with internet access who follow city social media and were online during workday hours, excluding working residents, elderly citizens without social media, and park visitors who do not use the platform.**

STEP-BY-STEP EXPLANATION

- College Board Big Idea 2 (DAT-2.E) emphasizes identifying **limitations and biases in data collection**:
- • A dataset is only as valid as its sampling methodology.
- • By posting exclusively on social media at 2:00 PM on a weekday, the sample systematically excluded individuals who work during that time, people who lack computers or internet access, and children or families who actively use the park but do not follow municipal government pages.
- This **sampling bias** means the dataset does not represent the broader community population, producing skewed and invalid policy conclusions.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Voting surveys store tallies in database rows, not overflowing registers that switch polarities.
✗ **(C):** Compression algorithms do not selectively target and delete specific voter opinions.
✗ **(D):** All web servers collect standard metadata (IP, timestamps); metadata does not invalidate surveys.

### Question 85: The 3 Vs of Big Data (Volume, Velocity, Variety)

★★★★ HIGH PRIORITY

**Correct Answer: (C) — Big Data, characterized by immense Volume (size), high Velocity (rapid influx), and diverse Variety (heterogeneous formats).**

STEP-BY-STEP EXPLANATION

- **Big Data** refers to datasets whose size, complexity, and speed of arrival exceed the capacity of traditional relational databases and single-server computing tools.
- Big data is classically defined by the '3 Vs':
- 1. **Volume:** Immense scale of data (terabytes, petabytes, exabytes).
- 2. **Velocity:** Extreme speed at which new data is generated and must be processed in real time.
- 3. **Variety:** Diverse, heterogeneous data types (structured tables, uncompressed sensor streams, satellite imagery, text logs).
- Analyzing Big Data requires distributed clusters and specialized parallel processing frameworks.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** RLE is a simple compression method, not a paradigm for planetary data streams.
✗ **(B):** Single-core processors cannot handle big data; petabyte processing requires massive parallelism.
✗ **(D):** Dictionary compression does not replace physical clouds with words.

### Question 86: Parallel and Distributed Processing for Big Data

★★★★ HIGH PRIORITY

**Correct Answer: (B) — Use parallel and distributed processing across a cluster of hundreds of interconnected cloud servers, dividing the dataset into chunks that are analyzed simultaneously.**

STEP-BY-STEP EXPLANATION

- Big data processing relies on **parallel and distributed computing**:
- • When a problem can be partitioned into independent tasks (such as analyzing different patient genomes or distinct chromosomes), the workload is distributed across hundreds or thousands of networked computing nodes.
- • Each server executes its assigned portion of the dataset concurrently (at the exact same time).
- • A cluster of 1,000 nodes working in parallel can complete a 12-year single-threaded computation in less than 5 days.
- This scalability is essential for modern bioinformatics, search engines, and scientific simulations.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** DNA nucleotide sequences (A, C, T, G) are vital biological code; lossy compression would destroy genetic data and ruin research.
✗ **(C):** ASCII encoding increases storage requirements (8 bits per letter) compared to 2-bit binary DNA encoding (00=A, 01=C, 10=G, 11=T).
✗ **(D):** Nucleotide base pairs are discrete chemical symbols, not floating-point numbers.

### Question 87: Outlier Detection and Handling

★★★★ HIGH PRIORITY

**Correct Answer: (B) — Identify the +999.0°C reading as an erroneous outlier caused by sensor hardware malfunction or default error code, and filter or flag it so it does not distort monthly average statistics.**

STEP-BY-STEP EXPLANATION

- An **outlier** is a data point that differs significantly from other observations in a dataset.
- In sensor data, extreme values like `999.0` or `-9999` are standard default error codes transmitted by microcontrollers when a temperature probe fails, loses power, or disconnects.
- If left in the dataset, this invalid reading would artificially skew the January average from -30°C to an absurd positive number.
- Data cleaning requires investigating outliers, identifying whether they represent genuine anomalies or equipment faults, and filtering out erroneous hardware spikes before analysis.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Deleting 30 days of valid scientific data because of 1 erroneous sensor reading is wasteful and unnecessary.
✗ **(C):** Meteorologists verify hardware validity before assuming miraculous physical events.
✗ **(D):** Blindly including obvious sensor error codes distorts analytical models and produces false conclusions.

### Question 88: Handling Missing Data in Large Datasets

★★★ MANDATORY

**Correct Answer: (A) — Discarding all 500 incomplete records ensures the remaining data is complete, but risks introducing bias if patients with missing blood pressure share common demographic or medical characteristics.**

STEP-BY-STEP EXPLANATION

- Handling missing data involves critical analytical trade-offs:
- • **Dropping records (Complete Case Analysis):** Discarding rows with missing fields is simple and avoids fabricating numbers. However, if the missingness is not purely random (e.g., if younger outpatient visitors were less likely to have blood pressure checked), removing them skews the remaining patient sample.
- • **Imputation:** Estimating missing values (using mean, median, or predictive models) preserves sample size, but introduces artificial variance.
- Data scientists must weigh these trade-offs based on the investigative domain.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Compression algorithms cannot predict or synthesize missing medical vitals.
✗ **(C):** Missing data directly impacts statistical calculations, either by altering sample size N or causing software calculation errors.
✗ **(D):** A blood pressure of 0 indicates clinical death; inserting 0 would catastrophically corrupt medical averages and risk patient safety.

### Question 89: Data Aggregation and Loss of Granularity

★★★★ HIGH PRIORITY

**Correct Answer: (D) — Aggregating data by taking a single citywide average (mean) masks localized inequality, because a massive wealth increase among a few ultra-wealthy households skewed the overall average upward.**

STEP-BY-STEP EXPLANATION

- **Data aggregation** summarizes detailed individual data points into high-level metrics (such as sums, means, or citywide averages).
- While aggregation simplifies visualization and reporting, it causes a significant **loss of granularity**:
- • The mean (average) is heavily sensitive to extreme positive outliers.
- • If a few billionaires or multimillionaires move into a city, their massive incomes pull the citywide arithmetic mean upward, even if median household income and the vast majority of residents saw their earnings decline.
- Reporting median income or neighborhood-level distributions provides a far more accurate representation than a single aggregated mean.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Averages frequently mask skewed distributions; mathematical means can easily mislead when variance is high.
✗ **(B):** Standard financial databases store numbers up to billions without overflowing at $1,000.
✗ **(C):** Income records are numeric database tables, not lossy JPEG photos.

### Question 90: Historical Bias and Algorithmic Fairness

★★★★★ VERY IMPORTANT

**Correct Answer: (C) — Historical bias in the training dataset: because past hiring practices favored male candidates, the algorithm learned and amplified historical human prejudice from the historical data.**

STEP-BY-STEP EXPLANATION

- College Board Big Idea 2 (DAT-2.E) explicitly addresses algorithmic bias and ethical data collection:
- • Machine learning models and automated data algorithms identify and reproduce patterns present in their training datasets.
- • If the historical training data reflects past human discrimination or systemic societal inequities (such as an industry historically hiring predominantly male engineers), the model learns that male gender is correlated with hiring success.
- • Consequently, the automated algorithm internalizes, perpetuates, and scales that historical human bias.
- This highlights the critical necessity of auditing, cleaning, and evaluating training data for fairness and representativeness.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Floating-point rounding errors do not generate complex gender discrimination in text resume evaluation.
✗ **(B):** ASCII and Unicode encode all names identically regardless of gender.
✗ **(D):** Network timeouts halt file transfers; they do not systematically score female candidates lower.

## Binary Sensor Representation & Integer Overflow in a Smart Greenhouse Monitor

**9 Points Total**

**OFFICIAL AP MODEL RESPONSE:**

• **(a) Binary to Decimal Conversion:**
- Positional place values for an 8-bit byte from left to right: 128, 64, 32, 16, 8, 4, 2, 1.
- Calculation: (1×128) + (1×64) + (1×32) + (1×16) + (1×8) + (1×4) + (1×2) + (1×1) = 255.
- Therefore, the binary value $11111111_2$ equals decimal 255.

• **(b) Register Overflow Mechanics:**
- Adding 1 to $11111111_2$ yields $100000000_2$ in standard 9-bit binary arithmetic.
- Because the microcontroller's register is physically constrained to 8 bits, the 9th bit (the 256 place value) cannot fit and is truncated/discarded.
- The 8 bits stored in memory roll over to $00000000_2$ (decimal 0). This is an **integer overflow error**.

• **(c) Operational Consequences on Irrigation:**
- If the overflow occurs when humidity is near 100% (or raw pulses exceed 255), the control algorithm reads the register as 0%.
- The greenhouse logic will interpret 0% as dangerously dry air and trigger an emergency watering cycle.
- This floods the crops with excessive water, potentially rotting root systems and ruining greenhouse yields.

• **(d) Prevention Strategies:**
- 1. *Wider Data Type:* Upgrade the register variable to a 16-bit unsigned integer (`uint16_t`), expanding capacity to 65,535 and preventing overflow.
- 2. *Input Clamping / Guard Rails:* Add defensive conditional checks in the update routine: `IF (counter >= 254) { counter <- 254 }` to clamp the counter at maximum capacity.

**OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):**

| Part | Scoring Criteria & Benchmark Expectation | Points |
|---|---|---|
| **(a)** | Correctly converts $11111111_2$ to 255 decimal using place value powers of 2 (128+64+32+16+8+4+2+1). | **2 pts** |
| **(b)** | Correctly identifies an integer overflow error and explains that the 9th carry bit is discarded, rolling the register over to $00000000_2$ (decimal 0). | **2 pts** |
| **(c)** | Explains the operational consequence: the system reads 0% humidity instead of maximum humidity, falsely concluding the air is bone-dry and unnecessarily flooding the greenhouse with sprinklers. | **3 pts** |
| **(d)** | Provides two valid technical solutions: upgrading the register to a 16-bit integer (capacity up to 65,535) and implementing software clamping/validation logic to cap values at 100%. | **2 pts** |

## Analog Audio Digitization, Sampling Rate & Bit Depth in a Voice Memo App

**9 Points Total**

**OFFICIAL AP MODEL RESPONSE:**

• **(a) Continuous Analog vs. Discrete Digital Audio:**
- The human voice generates an analog sound wave: a continuous variation in air pressure over time with an infinite number of possible frequencies and amplitudes.
- NoteVoice converts this into digital data by sampling the wave at fixed discrete time intervals and quantizing each voltage reading into a discrete binary number.

• **(b) File Size Calculations (10 minutes = 600 seconds):**
- *Formula:* Total Bits = Sampling Rate (Hz) × Bit Depth (bits) × Channels × Time (seconds).
- *Configuration Alpha:*
8,000 samples/sec × 8 bits/sample × 1 channel × 600 sec = **38,400,000 bits** (4.8 Megabytes).
- *Configuration Beta:*
44,100 samples/sec × 16 bits/sample × 2 channels × 600 sec = **846,720,000 bits** (105.84 Megabytes).

• **(c) Auditory Quality Differences:**
- *Sampling Rate:* Beta (44.1 kHz) captures frequencies up to ~22 kHz, reproducing crisp high-frequency consonants (s, t, f) naturally, whereas Alpha (8 kHz) caps at 4 kHz, sounding muffled like an old telephone.
- *Bit Depth:* Beta (16 bits = 65,536 levels) provides high dynamic range with near-zero background hiss, while Alpha (8 bits = 256 levels) introduces noticeable quantization noise.

• **(d) Trade-off Recommendation:**
- For students on tight cellular data caps, **Configuration Alpha** (or a mono 16 kHz compromise) is strongly recommended.
- Alpha's file is 22 times smaller (~4.8 MB vs ~105.8 MB), downloading almost instantly without exhausting mobile data, while human speech remains fully intelligible.

**OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):**

| Part | Scoring Criteria & Benchmark Expectation | Points |
|---|---|---|
| **(a)** | Explains that the analog sound wave is continuous with infinite intermediate volume/frequency states, while digital data consists of discrete numeric samples recorded at fixed intervals and quantized to binary levels. | **2 pts** |
| **(b)** | Correctly calculates both sizes: Configuration Alpha = 8,000 × 8 × 1 × 600 = 38,400,000 bits (~4.8 MB); Configuration Beta = 44,100 × 16 × 2 × 600 = 846,720,000 bits (~105.84 MB). | **3 pts** |
| **(c)** | Explains that Beta captures higher audio frequencies (natural voice clarity without muffling) due to 44.1 kHz sampling rate, and provides far wider dynamic volume range with reduced quantization noise due to 16-bit depth (65,536 levels vs 256). | **2 pts** |
| **(d)** | Justifies Configuration Alpha for low-bandwidth cellular environments because it is over 22 times smaller (~4.8 MB vs ~105.8 MB), making it fast to upload and stream over mobile data caps while maintaining intelligible speech. | **2 pts** |

## Run-Length Encoding vs. Dictionary-Based Lossless Compression in a Hospital Archive

**9 Points Total**

**OFFICIAL AP MODEL RESPONSE:**

• **(a) Lossless Definition & Healthcare Imperative:**
- A compression algorithm is *lossless* if the decompressed output is 100% bit-for-bit identical to the original uncompressed input.
- Lossless compression is strictly mandatory in medical archives because lossy compression permanently discards fine detail, which could erase a hairline bone fracture in an X-ray or alter medication dosages in a medical chart, causing fatal diagnostic errors.

• **(b) RLE Encoding and Reduction:**
- Original string: `BBBBBBBBBBWWWWBBBBBB` (20 characters).
- Run 1: 10 'B's -> `10B`.
- Run 2: 4 'W's -> `4W`.
- Run 3: 6 'B's -> `6B`.
- Compressed string: `10B4W6B` (6 characters without spaces).
- Percentage reduction: (20 − 6) / 20 × 100% = **70.0% reduction** (or 65% if formatted with single spaces `10B 4W 6B`).

• **(c) Why RLE Succeeds on Category 1 but Fails on Category 2:**
- Category 1 (dental scans) features massive contiguous blocks of identical color pixels (hundreds of identical black pixels in empty background space), which RLE compresses into a few bytes.
- Category 2 (English text) contains words where letters rarely repeat consecutively. In English sentences, runs are almost all length 1 (e.g., 'd-o-c-t-o-r' becomes `1d1o1c1t1o1r`), doubling the file size rather than compressing it.

• **(d) How Dictionary Compression Handles Category 2:**
- Dictionary compression (LZW) searches for multi-character patterns that recur across the document, regardless of whether they are adjacent.
- Phrases like 'patient tolerated procedure well' (32 characters) are assigned a short numeric pointer (e.g., Token #12). Every occurrence is replaced by Token #12.
- Upon decompression, the table reconstructs the exact original clinical phrasing with zero lost information.

**OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):**

| Part | Scoring Criteria & Benchmark Expectation | Points |
|---|---|---|
| **(a)** | Defines lossless compression as 100% reversible without bit loss, and explains that lossy compression could erase subtle dental fractures or alter medical dosages, risking patient health and legal compliance. | **2 pts** |
| **(b)** | Correctly encodes `BBBBBBBBBBWWWWBBBBBB` as `10B 4W 6B` (or `10B4W6B` / 6 chars) and calculates reduction: (20 - 6) / 20 = 70% reduction (or 7 chars with spaces = 65%). | **3 pts** |
| **(c)** | Explains that X-rays feature long contiguous runs of identical pixels which RLE excels at, whereas English text rarely has consecutive identical letters (e.g., 'patient' has runs of length 1), causing RLE to expand text size. | **2 pts** |
| **(d)** | Explains that dictionary compression identifies multi-word repeated phrases throughout the text, replacing them with compact numeric tokens in a lookup table, preserving exact words bit-for-bit upon decompression. | **2 pts** |

## Lossy JPEG Compression & Quality Trade-Offs in an Online School Newspaper

**9 Points Total**

**OFFICIAL AP MODEL RESPONSE:**

• **(a) Compression Type & Data Fate:**
- JPEG is a **lossy compression** algorithm.
- During conversion, mathematical transformations (Discrete Cosine Transform and quantization) identify and permanently discard subtle color variations and high-frequency edge textures.
- This discarded data is permanently deleted from the file and cannot be recovered.

• **(b) Compression Ratio & Space Savings Calculations:**
- *Compression Ratio:* Uncompressed Size / Compressed Size = 25 MB / 1.2 MB ≈ **20.83 : 1**.
- *Percentage Space Saved:* ((25 − 1.2) / 25) × 100% = (23.8 / 25) × 100% = **95.2% storage reduction**.

• **(c) Psychovisual Principles Exploited by JPEG:**
- 1. *Luminance vs. Chrominance Sensitivity:* Human eyes possess far more rod cells (detecting brightness) than cone cells (detecting color). JPEG preserves luminance detail while heavily subsampling color (chroma subsampling), which the human visual cortex barely notices.
- 2. *High-Frequency Insensitivity:* The human eye cannot perceive rapid spatial fluctuations in color across tiny distances. JPEG rounds off high-frequency coefficients to zero with minimal subjective loss.

• **(d) Artifacts & Irreversibility:**
- *Artifact:* At 5% quality, visible **macroblocking** (an 8×8 square pixel grid pattern) and blurry 'ringing' halos around sharp edges will appear.
- *Irreversibility:* The 25 MB original cannot be restored from the 1.2 MB JPEG because the discarded bits were permanently omitted during quantization; decompressing merely renders the approximated data.

**OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):**

| Part | Scoring Criteria & Benchmark Expectation | Points |
|---|---|---|
| **(a)** | Correctly identifies JPEG as lossy compression and explains that high-frequency color and gradient data is permanently discarded and cannot be recovered. | **2 pts** |
| **(b)** | Correctly computes ratio: 25 / 1.2 ≈ 20.8:1, and percentage reduction: (25 - 1.2) / 25 = 95.2% space reduction. | **2 pts** |
| **(c)** | Identifies two valid psychovisual mechanisms: human eyes are far more sensitive to luminance (brightness) than chrominance (color), and human retinas cannot detect subtle high-frequency spatial texture variations. | **3 pts** |
| **(d)** | Identifies macroblocking (8x8 pixel blocks) or ringing halos, and explains irreversibility: discarded bits no longer exist anywhere in storage. | **2 pts** |

## EXIF Photo Metadata Extraction & Privacy Protection in a Student Photo Gallery

9 Points Total

### OFFICIAL AP MODEL RESPONSE:

• **(a) Metadata Definition vs. Primary Data:**
- *Metadata* is structured 'data about data' that provides administrative, technical, and contextual details describing a digital file.
- The *metadata* shown consists of attributes like camera model, shutter speed, and timestamp. The *primary image data* consists of the actual 2D matrix of pixel RGB color values that make up the visual picture.

• **(b) Privacy & Security Risks:**
- 1. `GPSLatitude` and `GPSLongitude`: Reveal the exact geographic location (down to a few meters) where the photo was taken. For photos taken at home, this exposes the student's residential address to strangers.
- 2. `DateTimeOriginal`: Pinpoints the exact minute the student was at that location, enabling predatory tracking of daily routines and travel schedules.

• **(c) Legitimate Functional Uses of Metadata:**
- 1. *Educational Critiques:* Club members can filter photos by ISO, aperture, and focal length to analyze camera techniques and learn photography skills.
- 2. *Chronological Sorting:* The gallery can automatically organize photos in seasonal timelines or album categories without manual user tagging.

• **(d) Sanitization (Scrubbing) Workflow:**
- 1. Upon upload, the backend software parses the file's EXIF header structure.
- 2. The script explicitly deletes or sets to null the GPS fields (`GPSLatitude`, `GPSLongitude`) and user identifying tags.
- 3. The cleaned image header is rewritten back to disk alongside the untouched pixel payload.
- 4. Because only header tags are removed without re-encoding pixels, image quality remains 100% unaffected.

### OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):

| Part | Scoring Criteria & Benchmark Expectation | Points |
|---|---|---|
| **(a)** | Defines metadata as 'data about data' providing context, and distinguishes that metadata describes file properties while primary data consists of the actual pixel color values. | **2 pts** |
| **(b)** | Identifies GPSLatitude and GPSLongitude (exact physical home location) and DateTimeOriginal (tracks student daily schedules/routines). | **2 pts** |
| **(c)** | Provides two legitimate use cases: sorting/filtering photos by camera equipment/ISO settings for photography critiques, and indexing/searching photos chronologically. | **2 pts** |
| **(d)** | Outlines the sanitization workflow: extract and strip/delete GPS and personal EXIF tags from the header before saving, without touching or re-compressing the pixel payload to avoid quality loss. | **3 pts** |

## Data Cleaning, Deduplication & Outlier Detection in a Municipal Health Survey

**9 Points Total**

**OFFICIAL AP MODEL RESPONSE:**

• **(a) Data Anomalies Identified:**
- 1. *Invalid Domain Outlier (Record 10422):* `ResidentAge = -8` is physically impossible for a living human (data entry error).
- 2. *Impossible Outlier (Record 10425):* `DailySleepHours = 99.0` is impossible in a 24-hour day (likely a default missing-value placeholder).
- 3. *Inconsistent Formatting:* `SurveyDate` uses three conflicting formats ('2026-10-12', '10/12/2026', 'Oct 12, 2026').
- 4. *Missing Data (Record 10426):* Blank sleep field.

• **(b) Duplicate Records Handling:**
- Records 10423 and 10424 have identical resident age (42), sleep (6.0), exercise (60), and date, likely caused by a resident double-clicking 'Submit'.
- Deduplication software identifies matching field signatures, retains the first record (10423), and deletes the redundant clone (10424) to avoid double-counting.

• **(c) Analysis of Record 10425 (`99.0` Sleep):**
- 99.0 hours of daily sleep is physically impossible. It is an erroneous outlier or a legacy code for 'refused to answer'.
- Leaving 99.0 in the dataset heavily skews the arithmetic mean upward, misleading public health officials into believing residents sleep far more than they actually do.

• **(d) Strategies for Blank Field (Record 10426):**
- *Strategy 1 (Removal):* Discard Record 10426 from sleep calculations. *Trade-off:* Ensures complete verified data, but loses the valid exercise data (20 minutes).
- *Strategy 2 (Mean/Median Imputation):* Replace [BLANK] with the city median sleep value (~7.0 hours). *Trade-off:* Retains the patient record for multi-variable analysis, but introduces an assumed estimate.

**OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):**

| Part | Scoring Criteria & Benchmark Expectation | Points |
|---|---|---|
| **(a)** | Identifies 3 valid anomalies: invalid negative age in Record 10422 (domain validation error), impossible sleep outlier of 99.0 in Record 10425 (sensor/entry error), duplicate row in Record 10424, or inconsistent date formatting. | **3 pts** |
| **(b)** | Explains that 10423 and 10424 are duplicate records caused by accidental double submission, and deduplication software removes the redundant record while keeping one unique row. | **2 pts** |
| **(c)** | Explains that 99.0 hours of sleep in a day is physically impossible (outlier/default error code); leaving it uncorrected falsely inflates the mean sleep average of the entire dataset. | **2 pts** |
| **(d)** | Evaluates dropping the record (Complete Case: clean data but loses exercise sample) vs. imputation (substituting median/mean: retains row but introduces artificial estimate). | **2 pts** |

## Floating-Point Round-Off Error & Financial Accumulation in a School Meal Account System

**9 Points Total**

**OFFICIAL AP MODEL RESPONSE:**

• **(a) Why Strict Equality Failed:**
- In binary floating-point representation, 0.10 and 0.20 cannot be stored with exact terminating precision.
- Their sum evaluates to approximately `0.30000000000000004`.
- Because `0.30000000000000004 == 0.30` evaluates to FALSE, the conditional check fails and branches to `logDiscrepancyNotice()`.

• **(b) Why 0.10 Repeats Infinitely in Binary:**
- In any base b, a fraction terminates only if the prime factors of its denominator divide base b.
- In base-2 (binary), the only prime factor is 2.
- The fraction 1/10 has prime factors 2 and 5. Because 5 does not divide 2, 1/10 produces an infinite repeating binary fraction (`$0.0001100110011\ldots_2$`) that must be truncated in finite memory bits.

• **(c) Cumulative Error Drift Over 500,000 Transactions:**
- While a single rounding error of $\sim 10^{-16}$ is unnoticeable in an individual cafeteria purchase, the ledger compounded this error across 500,000 transactions.
- Compounded over a full academic year, these tiny fractional discrepancies accumulated into a real $43.12 imbalance in the general financial ledger.

• **(d) The Integer-Cents Solution:**
- *Strategy:* Store all monetary values as **whole integers representing cents** instead of floating-point dollars.
- *Rewritten Script:*
`itemPrice1 <- 10 // 10 cents`
`itemPrice2 <- 20 // 20 cents`
`totalCharged <- itemPrice1 + itemPrice2 // exactly 30 cents`
`IF (totalCharged == 30) { applyStudentDiscount() }`
- Because integer addition is 100% exact in computer hardware, round-off error is completely eliminated.

**OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):**

| Part | Scoring Criteria & Benchmark Expectation | Points |
|---|---|---|
| **(a)** | Explains that in binary floating point, 0.10 + 0.20 evaluates to ~0.30000000000000004, causing strict equality `== 0.30` to evaluate to FALSE and trigger the discrepancy notice. | **2 pts** |
| **(b)** | Explains that fractions only terminate in binary if denominators are powers of 2; because 1/10 has a factor of 5, it forms an infinite repeating binary fraction that must be truncated. | **2 pts** |
| **(c)** | Explains error accumulation: tiny rounding discrepancies of $10^{-16}$ compounded across 500,000 successive addition and subtraction operations into a multi-dollar ledger error. | **2 pts** |
| **(d)** | Proposes integer-cents representation: store money as integer cents (e.g., $0.10 is 10 cents, $0.20 is 20 cents), where 10 + 20 == 30 evaluates exactly to TRUE with zero round-off. | **3 pts** |

## Hexadecimal Color Representation & Bitwise RGB Channels in an Accessibility Screen Tint Tool

**9 Points Total**

**OFFICIAL AP MODEL RESPONSE:**

• **(a) Hexadecimal to Decimal Conversion for `#9B30FF`:**
- *Red Channel (`$9B_{16}$`):*
In hex, B = 11. Red = (9 × 16) + (11 × 1) = 144 + 11 = **155**.
- *Green Channel (`$30_{16}$`):*
Green = (3 × 16) + (0 × 1) = 48 + 0 = **48**.
- *Blue Channel (`$FF_{16}$`):*
In hex, F = 15. Blue = (15 × 16) + (15 × 1) = 240 + 15 = **255**.
- Resulting RGB tuple: (155, 48, 255) — a vibrant violet/purple.

• **(b) Binary Conversion of Red Channel (`$9B_{16}$`):**
- Convert each hex digit to a 4-bit nibble:
- $9_{16}$ in binary = `$1001_2$` (8 + 1).
- $B_{16}$ (11) in binary = `$1011_2$` (8 + 2 + 1).
- Combine the nibbles: **$10011011_2$**.

• **(c) Why Hexadecimal is Preferred Over Binary:**
- 24-bit binary color codes require 24 binary digits (e.g., `$100110110011000011111111_2$`), which is clumsy, error-prone, and difficult for humans to read or type.
- Because $16 = 2^4$, each hexadecimal digit represents exactly 4 bits, allowing 1 full byte to be expressed in exactly 2 hex characters.
- Hexadecimal compresses 24 bits into a compact 6-character string (`#9B30FF`) while maintaining a direct, unambiguous mapping to hardware color registers.

• **(d) Black and White in the Additive RGB Model:**
- `#000000` sets Red = 0, Green = 0, Blue = 0, turning off all sub-pixel light emitters on the display and producing pure **black**.
- `#FFFFFF` sets Red = 255, Green = 255, Blue = 255 (maximum intensity). In additive color physics, combining full-intensity red, green, and blue light wavelengths stimulates human retinal photoreceptors simultaneously, creating the perception of pure **white**.

**OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):**

| Part | Scoring Criteria & Benchmark Expectation | Points |
|---|---|---|
| **(a)** | Correctly converts all 3 channels: Red = $9B_{16}$ = (9×16)+11 = 155; Green = $30_{16}$ = (3×16)+0 = 48; Blue = $FF_{16}$ = (15×16)+15 = 255. | **3 pts** |
| **(b)** | Correctly converts $9B_{16}$ to 8-bit binary: $9_{16}$ = $1001_2$, $B_{16}$ (11) = $1011_2$ -> $10011011_2$. | **2 pts** |
| **(c)** | Explains that hex provides a compact 6-character human-readable notation where each hex digit maps cleanly to 4 bits (2 hex digits per byte), avoiding unwieldy 24-bit binary strings. | **2 pts** |
| **(d)** | Explains that #000000 turns off all subpixels producing black, while #FFFFFF illuminates Red, Green, and Blue subpixels at full intensity (255), which additive light physics combines into white. | **2 pts** |

## Big Data Parallel Processing & Correlation vs. Causation in an Academic Analytics Study

**9 Points Total**

### OFFICIAL AP MODEL RESPONSE:

• **(a) The 3 Vs of Big Data Exhibited:**
- *Volume:* 10 Terabytes of cumulative data tracking 40,000 students over four years.
- *Velocity:* High-speed streaming data generated continuously every second as thousands of mobile devices authenticate to campus Wi-Fi access points.
- *Variety:* Heterogeneous data formats combining network IP logs, web LMS clickstreams, physical RFID card swipes, and academic letter grades.

• **(b) Necessity of Parallel and Distributed Computing:**
- A single computer workstation has limited RAM and CPU cores; processing 10 TB sequentially would require weeks or months and risk out-of-memory crashes.
- A distributed cluster (e.g., using MapReduce or Apache Spark) partitions the 10 TB dataset across dozens of interconnected servers, allowing each node to process a subset of students concurrently in parallel, completing the analysis in hours.

• **(c) Correlation vs. Causation Fallacy:**
- Concluding that late-night Wi-Fi causes higher GPAs confuses **correlation with causation**.
- Connecting to library Wi-Fi is correlated with a third *confounding variable*: academic motivation, conscientious study habits, and time investment.
- Simply forcing students to connect their phones to library Wi-Fi after 11 PM without studying will not raise their grades.

• **(d) Student Privacy and Surveillance Risks:**
- 1. *Physical Location Tracking:* Wi-Fi access points record exact physical campus locations, allowing administrators or bad actors to track students' intimate daily movements, dorm rooms, and medical center visits.
- 2. *Inferred Social Associations:* Analyzing co-located Wi-Fi timestamps can map student dating relationships, political club attendance, and private associations without explicit consent.

### OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):

| Part | Scoring Criteria & Benchmark Expectation | Points |
|---|---|---|
| **(a)** | Explains Volume (10 Terabytes across 40k students), Velocity (real-time continuous second-by-second Wi-Fi logs and clickstreams), and Variety (heterogeneous data: network logs, LMS clicks, swipe transactions, GPA numbers). | **3 pts** |
| **(b)** | Explains parallel processing: a single computer would take months/years and run out of RAM, whereas parallel cluster nodes divide data into independent chunks processed concurrently in minutes. | **2 pts** |
| **(c)** | Explains correlation does not equal causation: late-night Wi-Fi connects correlate with conscientious academic work ethic/study habits (confounding variable); Wi-Fi radio waves do not cause higher intelligence. | **2 pts** |
| **(d)** | Identifies two privacy risks: reveals intimate physical locations/dorm visits/movement tracking, and exposes personal social associations without student consent. | **2 pts** |

## Comprehensive Data Pipeline: From Analog Environmental Sampling to Lossless Cloud Telemetry

**9 Points Total**

**OFFICIAL AP MODEL RESPONSE:**

• **(a) Analog-to-Digital Conversion & Quantization Levels:**
- Ocean temperature varies continuously over time with infinite analog states. The ADC samples this continuous voltage once every 10 seconds and quantizes the amplitude to the nearest discrete step.
- A 10-bit ADC provides $2^{10}$ = **1,024 distinct quantization levels** (representing values from $0000000000_2$ to $1111111111_2$, decimal 0 to 1,023).

• **(b) Run-Length Encoding Representation:**
- Original sequence: `18, 18, 18, 18, 18, 18, 19, 19, 19, 18, 18, 18` (12 readings).
- Run 1: Six 18s -> `6(18)`.
- Run 2: Three 19s -> `3(19)`.
- Run 3: Three 18s -> `3(18)`.
- Compressed stream: **`6(18) 3(19) 3(18)`** (or `6:18, 3:19, 3:18`).

• **(c) Why Lossless Compression is Essential:**
- Oceanographic research and climate modeling require 100% exact numerical accuracy to detect minute fractions of a degree in ocean warming.
- Lossy compression permanently discards data and introduces rounding artifacts that would falsify scientific measurements.
- Lossless RLE shrinks repetitive night-time readings without losing a single fraction of a degree upon decompression.

• **(d) Metadata Header vs. Payload in Satellite Telemetry:**
- *Payload:* The compressed temperature readings representing the actual ocean measurements collected by the buoy.
- *Metadata Header:* The 32-byte contextual data packet containing the buoy's identity ID, GPS coordinates, timestamp, and battery voltage.
- *Station Processing:* The satellite receiving station uses the **metadata** to confirm the buoy's location, verify battery health, and catalog the observation time, and uses the **payload** to decompress and plot the temperature time series into the scientific climate database.

**OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):**

| Part | Scoring Criteria & Benchmark Expectation | Points |
|---|---|---|
| **(a)** | Explains analog voltage sampling at discrete 10-second intervals and quantization to $2^{10}$ = 1,024 distinct discrete binary voltage levels. | **2 pts** |
| **(b)** | Correctly encodes the sequence: `6(18) 3(19) 3(18)` or `6,18,3,19,3,18` without data loss. | **2 pts** |
| **(c)** | Explains that scientific climate models require exact temperatures bit-for-bit; lossy compression would corrupt decimal fractions, destroying long-term climate trend analysis. | **2 pts** |
| **(d)** | Distinguishes that metadata (32-byte header: GPS, timestamp, battery) provides routing/sensor health context while payload contains actual temperature numbers; satellite uses metadata to verify buoy identity/location and payload for scientific modeling. | **3 pts** |

# Essential Unit 2 Data Representation Reference Framework

Core powers of 2 table, hexadecimal equivalents, sampling equations, compression taxonomies, and metadata definitions.

- **Powers of 2 and Bit Representation:**
  • $2^0$=1, $2^1$=2, $2^2$=4, $2^3$=8, $2^4$=16, $2^5$=32, $2^6$=64, $2^7$=128, $2^8$=256, $2^9$=512, $2^{10}$=1024, $2^{16}$=65,536, $2^{32}$=4,294,967,296.
  • An unsigned sequence of n bits can represent exactly $2^n$ distinct values, ranging from 0 to $2^n - 1$.
  • Every additional bit doubles the total quantity of distinct representable values.
- **Hexadecimal (Base-16) & Text Encoding:**
  • Digits: 0–9 and A (10), B (11), C (12), D (13), E (14), F (15). Exactly 4 bits (1 nibble) map to 1 hex digit. 2 hex digits = 1 byte (8 bits).
  • 24-bit RGB Color: `#RRGGBB` (Red, Green, Blue each $00_{16}$ to $FF_{16}$, decimal 0 to 255).
  • ASCII: 7-bit standard (128 characters). Unicode (UTF-8/UTF-16): universal standard supporting >1.1 million characters, international languages, and emojis.
- **Integer Overflow vs. Floating-Point Round-Off:**
  • *Integer Overflow:* Occurs when an integer calculation yields a value too large for the allocated fixed-width bits, causing rollover/wraparound (e.g., 8-bit unsigned 255 + 1 = 0).
  • *Floating-Point Round-Off:* Occurs because non-terminating fractional real numbers (like 0.1 and 0.2) cannot be represented with infinite precision in finite binary bits, creating tiny precision discrepancies.
- **Analog vs. Digital & Audio Sampling:**
  • Analog is continuous; Digital is discrete (sampled approximations represented in binary).
  • Sampling Rate (Hz): Number of samples measured per second. Bit Depth: Number of bits per sample (quantization resolution).
  • `Audio Size (bits) = Sample Rate (Hz) × Bit Depth (bits) × Channels × Time (seconds)`.
- **Data Compression & Metadata:**
  • *Lossless Compression (ZIP, PNG, FLAC, RLE, LZW):* 100% reversible bit-for-bit without data loss. Required for code, text, and financial/medical data.
  • *Lossy Compression (JPEG, MP3, MP4):* Permanently discards imperceptible or redundant data based on human psychovisuals/psychoacoustics. High compression ratios (10:1 to 20:1), but irreversible.
  • *Metadata:* Data about data (timestamps, GPS, file size, author). Changing metadata does not alter primary content. Poses significant privacy risks if not scrubbed.

**Disclaimer:** This educational publication contains original AP Computer Science Principles-aligned practice questions developed strictly for pedagogical preparation and exam mastery. It is not affiliated with, endorsed by, or approved by the College Board. "AP" and "Advanced Placement" are registered trademarks of the College Board.



# Thank You, Future Data Scientists!

**DEDICATED TO ALL STUDENTS STRIVING FOR AP EXCELLENCE**

You have successfully completed all 100 master questions of the **AP Computer Science Principles Unit 2 Master Set**. By mastering positional binary arithmetic, hexadecimal color spaces, register overflow limits, floating-point round-off, continuous-to-discrete sampling, lossless Run-Length Encoding, lossy perceptual codecs, and ethical metadata privacy, you have unlocked the language that powers the global digital universe.

> **"The universe may be continuous and analog, but through the beauty of binary abstraction, we capture, preserve, and compute reality — one bit at a time."**

**BEST WISHES FOR A SCORE OF 5 ON THE AP COMPUTER SCIENCE PRINCIPLES EXAM!**